TASK 1 — Create VPC with 2 Public + 2 Private Subnets

Outcome

A fully segmented private network in AWS with 4 subnets across 2 AZs and an Internet Gateway attached — ready for routing and EC2 deployment.

Objective

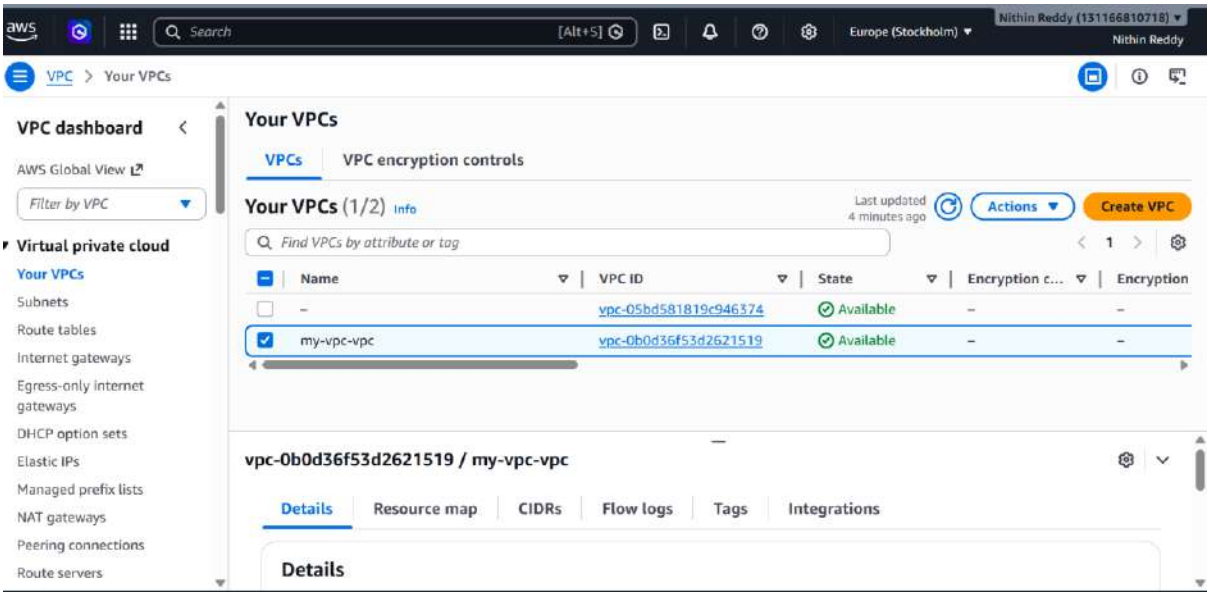
Create an isolated AWS network (VPC) with CIDR 10.0.0.0/16, divided into 4 subnets across 2 Availability Zones — 2 public (internet-facing) and 2 private (internal only).

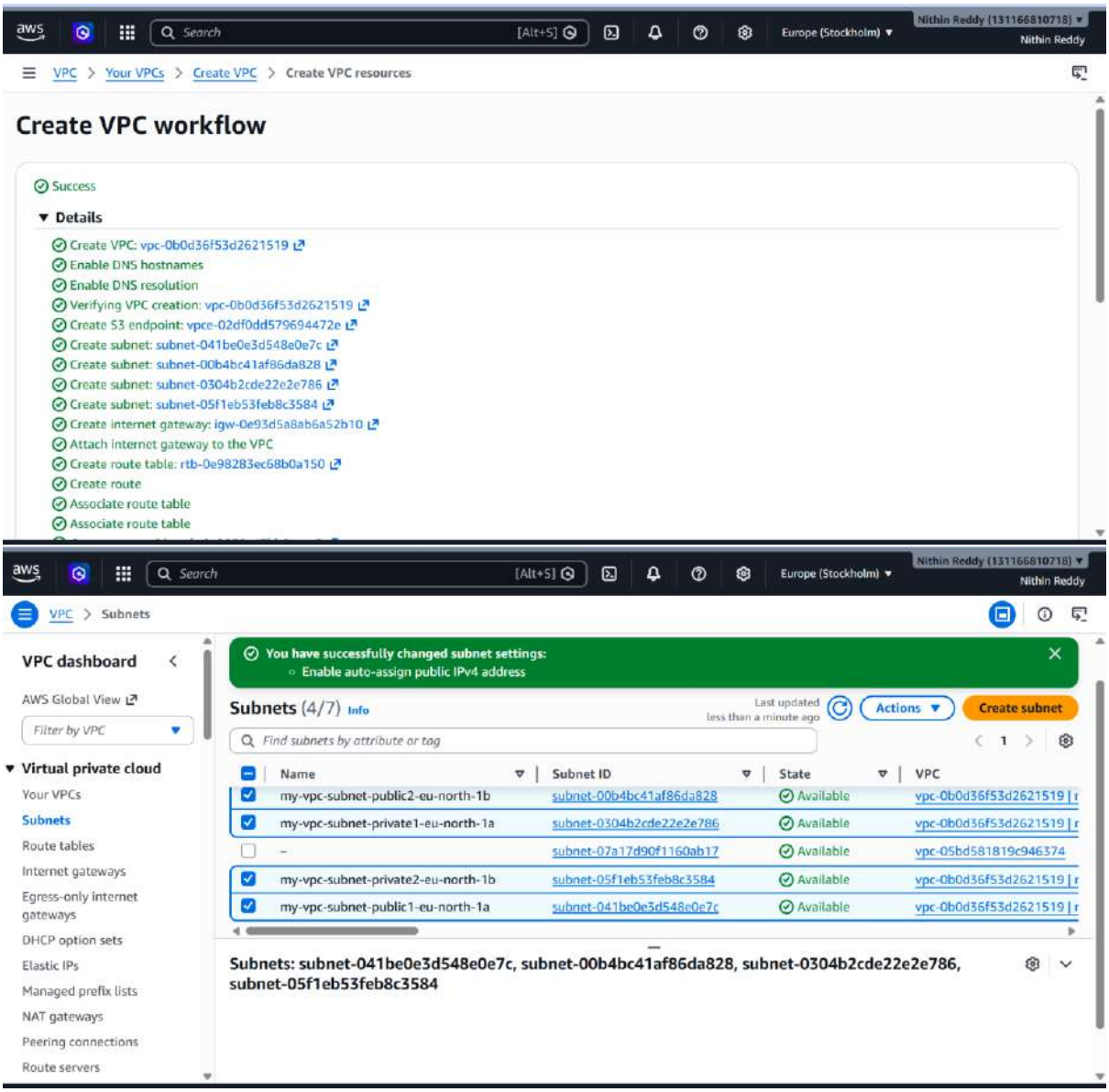
Why This Task?

A VPC is your private data center inside AWS. Without it, you can't deploy anything. Subnets divide your network into zones — public subnets for web servers, private subnets for databases/app servers. Using 2 AZs gives you High Availability.

Prerequisites

- AWS account with admin/VPC permissions
- Region selected (use **ap-south-1** throughout)
- No prior VPC needed — starting fresh





Steps

Step 1 — Create the VPC

1. Go to **AWS Console → VPC → Your VPCs → Create VPC**
2. Select **"VPC Only"**
3. Fill in:
    - Name: my-vpc
    - IPv4 CIDR: 10.0.0.0/16
    - IPv6: No IPv6

- Tenancy: Default

4. Click **Create VPC**
5. 📸 Screenshot: VPC created with State = available

Step 2 — Create Internet Gateway

1. VPC Console → **Internet Gateways → Create Internet Gateway**
2. Name: my-igw → Create
3. Select my-igw → **Actions → Attach to VPC** → select my-vpc → Attach
4. 📸 Screenshot: IGW State = attached

Step 3 — Create 4 Subnets

Go to **VPC → Subnets → Create Subnet** → Select VPC: my-vpc

Create all 4 one by one:

Subnet Name	AZ	CIDR
Public-Sub-1a	ap-south-1a	10.0.1.0/24
Public-Sub-1b	ap-south-1b	10.0.2.0/24
Private-Sub-1a	ap-south-1a	10.0.3.0/24
Private-Sub-1b	ap-south-1b	10.0.4.0/24

4. 📸 Screenshot: All 4 subnets visible in subnet list under my-vpc

Validation

- VPC my-vpc shows State = **available**, CIDR = 10.0.0.0/16
- 4 subnets appear in subnet list, all under my-vpc
- Internet Gateway my-igw shows State = **attached**
- Each subnet is in the correct AZ and CIDR block

Common Errors & Fixes

Error	Cause	Fix
CIDR block already exists	Another VPC has same range	Use 10.1.0.0/16 instead
IGW already attached	Only 1 IGW per VPC allowed	Detach old IGW first
Subnet CIDR conflict	Overlapping subnet ranges	Make sure each /24 is unique

TASK 2 — Enable DNS Hostname in VPC

Outcome

The VPC now supports DNS hostname assignment. Every EC2 instance launched in a public subnet will automatically receive a resolvable public DNS name.

Objective

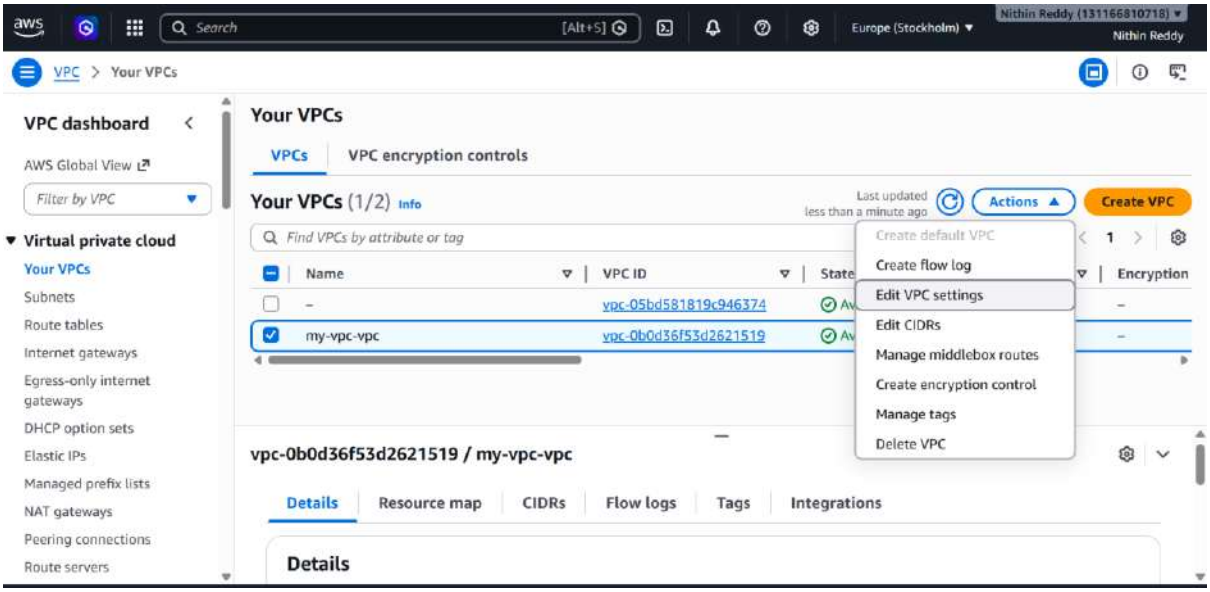
Enable DNS hostnames on my-vpc so that EC2 instances receive a public DNS name like ec2-13-x-x-x.ap-south-1.compute.amazonaws.com automatically.

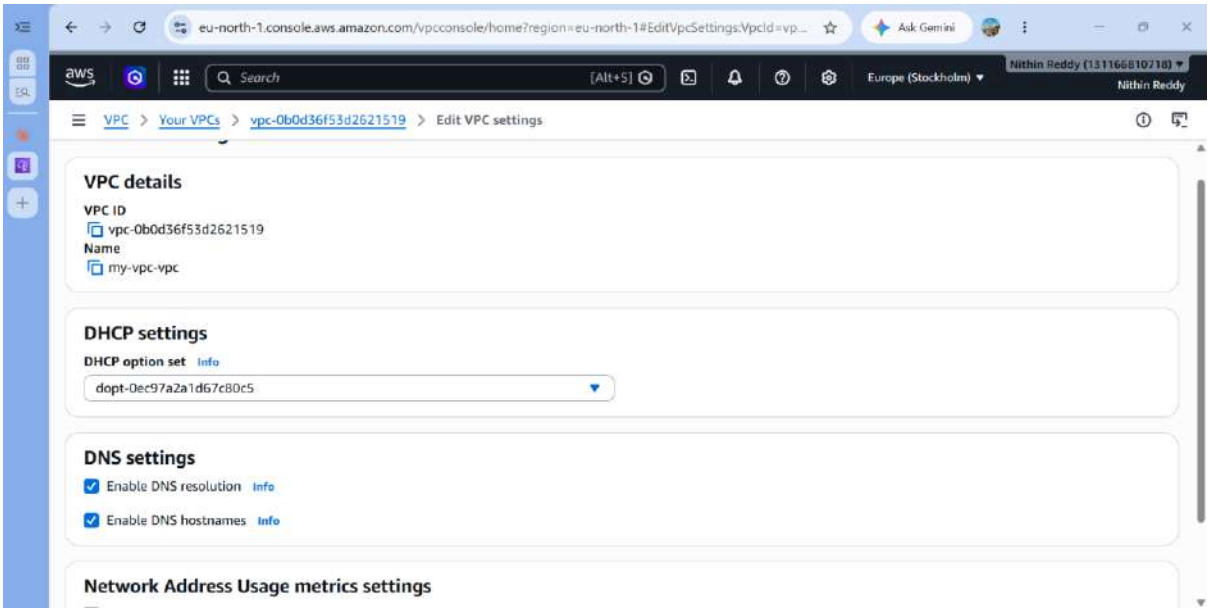
Why This Task?

Custom VPCs have DNS hostnames **disabled by default**. Without it, your EC2 instances won't get a public DNS name — you'd only have a raw IP, which breaks many services (ELB health checks, RDS endpoints, internal service discovery). Enabling this is a mandatory step before launching EC2s.

Prerequisites

- Task 1 complete — my-vpc exists





Steps

- **Step 1**
- Go to **VPC → Your VPCs** → select my-vpc
- Click **Actions → Edit VPC Settings**
- Under DNS Settings, tick:
- ✅ **Enable DNS resolution** (should already be on)
- ✅ **Enable DNS hostnames** ← this is what you're enabling
- Click **Save**

Screenshot: Both DNS options showing **Enabled**

- **Validation**
- VPC Settings shows DNS hostnames = **Enabled**
- CLI returns EnableDnsHostnames: true
- After launching an EC2 later, it will have a Public DNS column filled in

Common Errors & Fixes

Error	Cause	Fix
Option greyed out	Insufficient IAM permissions	Ensure ec2:ModifyVpcAttribute permission
EC2 still has no DNS name after	DNS resolution also disabled	Make sure BOTH resolution + hostnames are enabled

TASK 3 — Enable Auto-Assign Public IP on Public Subnets

Objective

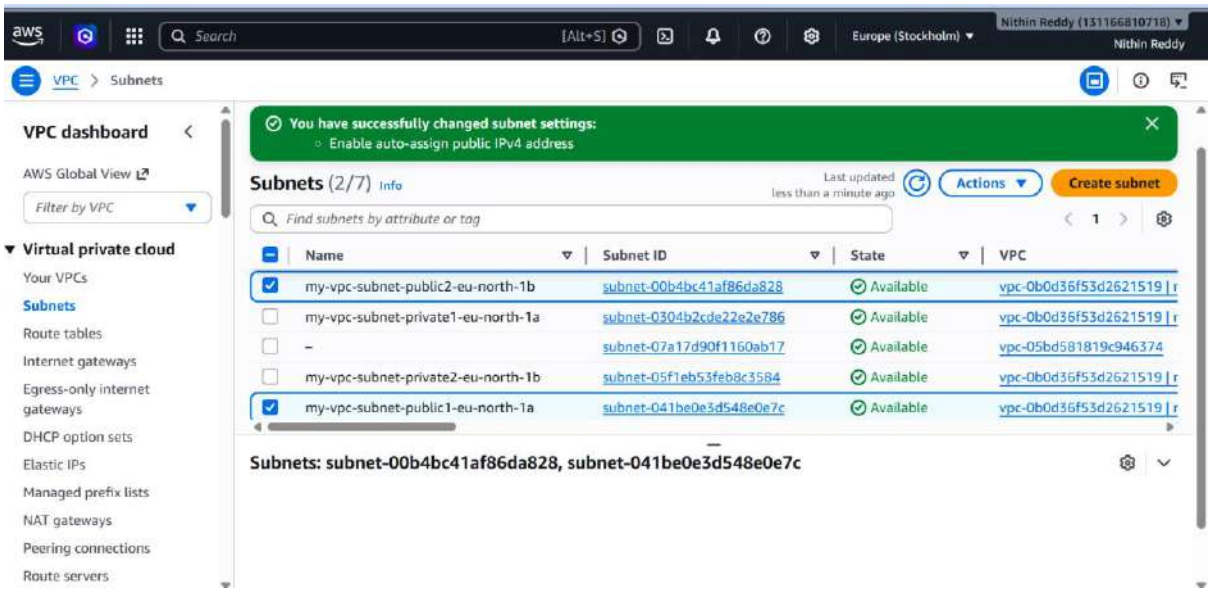
Configure Public-Sub-1a and Public-Sub-1b to automatically assign a public IPv4 address to any EC2 launched inside them — without needing to attach an Elastic IP manually.

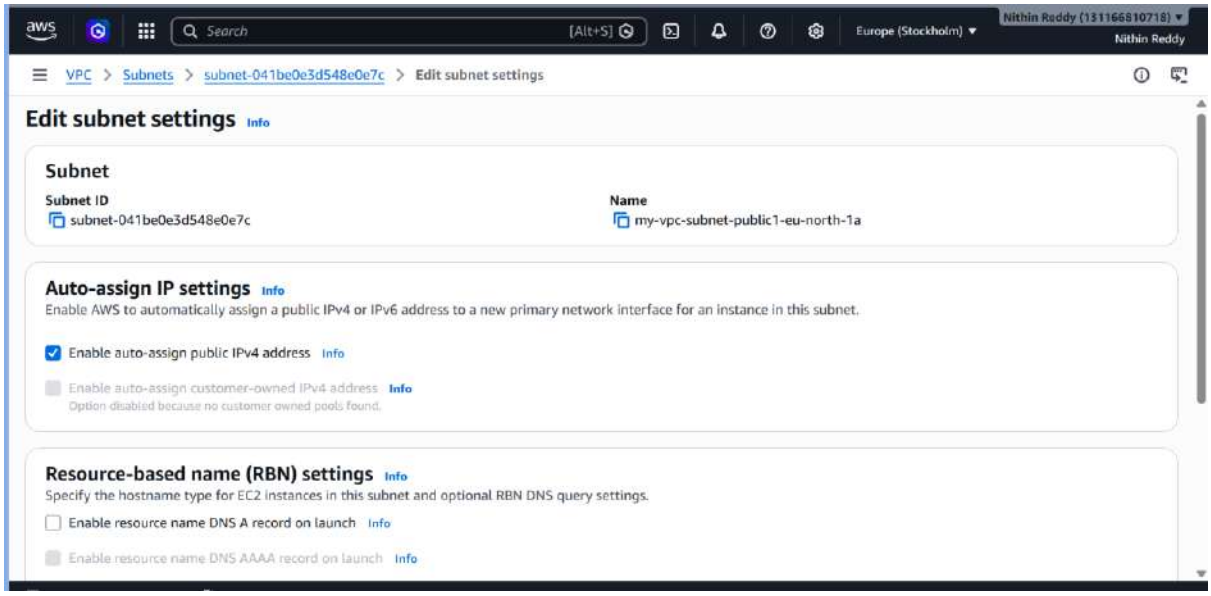
Why This Task?

Without this setting, EC2 instances in public subnets only get a private IP. They'd be invisible to the internet. Enabling auto-assign at the subnet level means every instance in that subnet automatically gets a public IP — this is what makes a subnet truly "public."

Prerequisites

- Tasks 1 & 2 complete





Steps

Step 1 — Enable on Public-Sub-1a

1. VPC Console → **Subnets** → select Public-Sub-1a
2. **Actions → Edit subnet settings**
3. Tick: ☑ **Enable auto-assign public IPv4 address**
4. Save

Step 2 — Enable on Public-Sub-1b

1. Repeat the same steps for Public-Sub-1b
2. 📸 Screenshot: Both public subnets showing **Auto-assign public IPv4 = Yes**

Validation

- Public-Sub-1a → Auto-assign public IPv4 = **Yes**
- Public-Sub-1b → Auto-assign public IPv4 = **Yes**
- Private-Sub-1a and Private-Sub-1b → Auto-assign = **No** (confirm this too)

Common Errors & Fixes

Error	Cause	Fix
EC2 still gets no public IP	Subnet setting saved but EC2 was launched before	Terminate and re-launch EC2
Accidentally enabled on private subnet	Human error	Go back and disable it immediately

TASK 4 — Add 2 Private Subnets to Private Route Table

Outcome

Private subnets are locked to internal-only routing. No inbound or outbound internet traffic is possible from private instances — the security boundary is enforced.

Objective

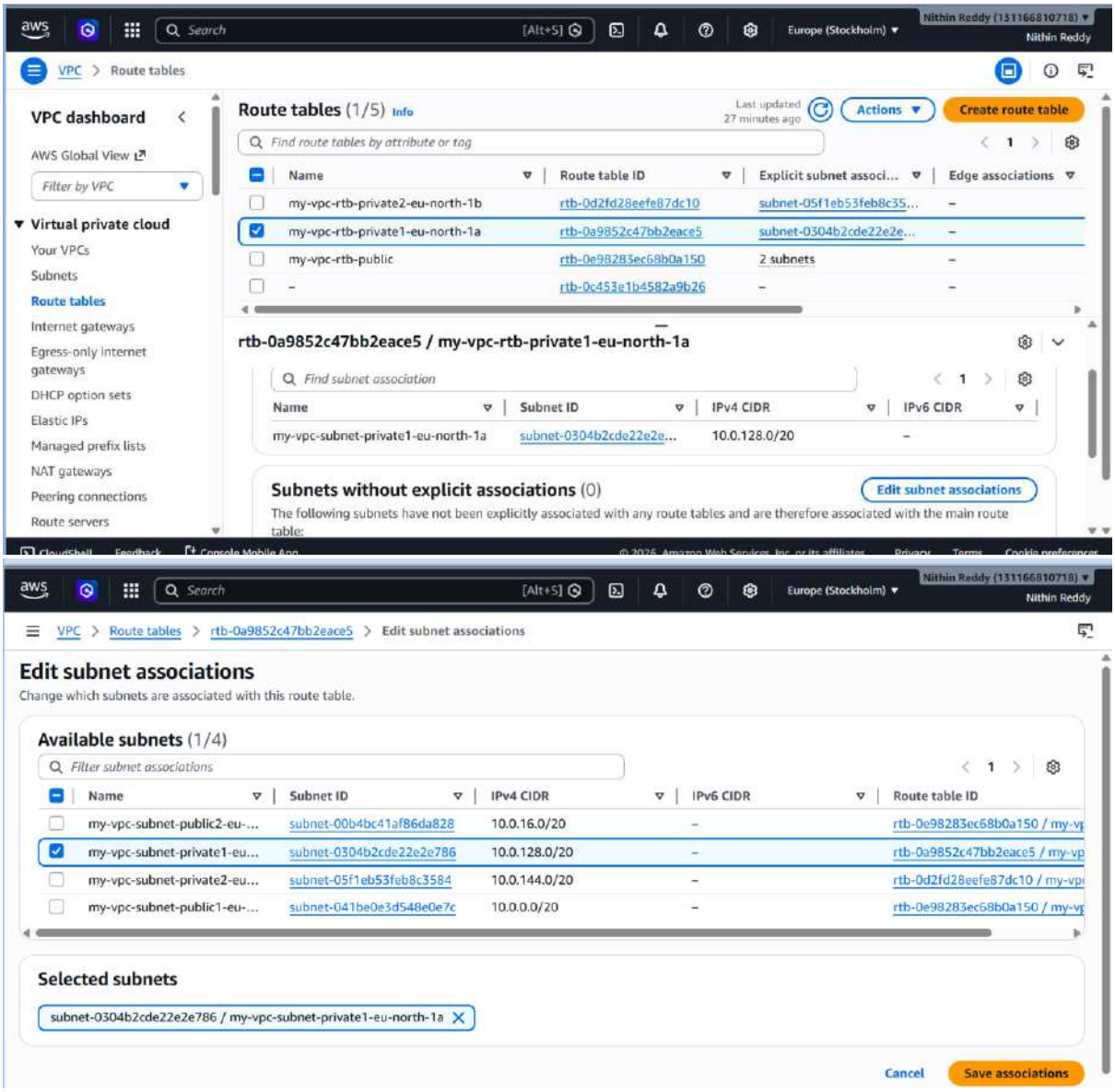
Create a dedicated **private route table**, associate Private-Sub-1a and Private-Sub-1b with it. This table will only allow internal VPC traffic — no internet.

Why This Task?

Route tables decide where traffic goes. A private route table with only a local route ensures private subnet instances can't reach or be reached from the internet. Without this, subnets fall back to the main route table — which might have an internet route and accidentally expose your private instances.

Prerequisites

- Task 1 complete — VPC and subnets exist



Steps

Step 1 — Create Private Route Table

1. VPC Console → **Route Tables → Create Route Table**
2. Name: private-rt
3. VPC: select my-vpc
4. Click **Create**

Step 2 — Associate Private Subnets

1. Select private-rt → click **Subnet Associations** tab

2. Click **Edit subnet associations**
3. Check both:
    - ☑ Private-Sub-1a
    - ☑ Private-Sub-1b
4. Click **Save associations**
5. 📸 Screenshot: private-rt showing both private subnets associated

Step 3 — Verify Routes

1. Click the **Routes** tab on private-rt
2. You should see exactly **1 route**:
    - 10.0.0.0/16 → local
3. 📸 Screenshot: Routes tab with only the local route

Validation

- private-rt created and visible in Route Tables
- Both private subnets are associated with private-rt
- Route table has **only 1 route**: 10.0.0.0/16 → local
- No 0.0.0.0/0 route exists (this confirms no internet access)

Common Errors & Fixes

Error	Cause	Fix
Subnet already associated with another RT	Main RT has explicit association	Remove it from old RT first
Route table shows 0.0.0.0/0	Wrong RT selected / IGW	Delete the 0.0.0.0/0

• **Error**	• **Cause**	• **Fix**
	route added by mistake	route from private-rt

TASK 5 Add 2 public subnets in public route table.

Outcome

A dedicated public route table exists and both public subnets are associated with it. The subnets are now ready to receive internet routing in Task 6.

Objective

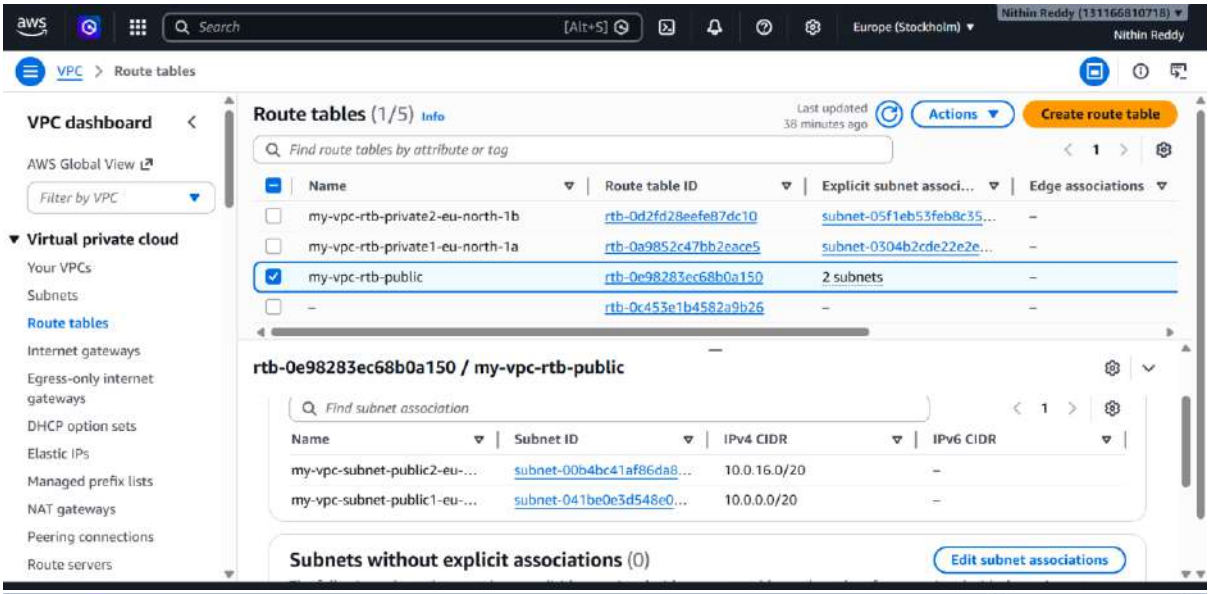
Create a **public route table**, associate both public subnets, and add a default internet route (0.0.0.0/0 → IGW). This is what truly makes subnets "public."

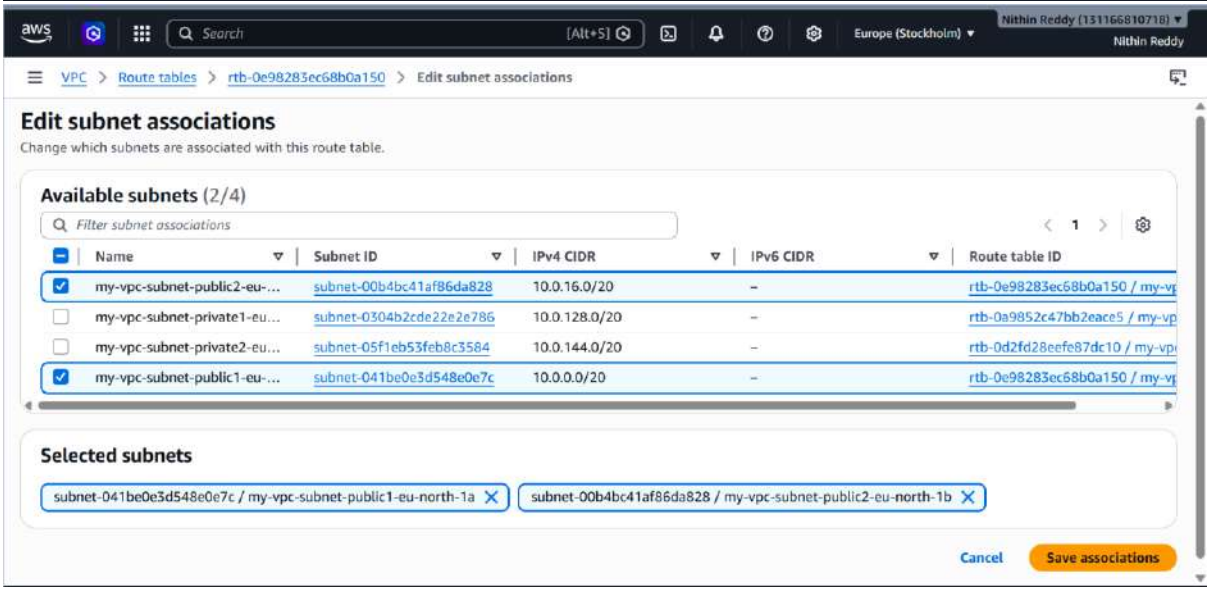
Why This Task?

Even with a public IP on your EC2 and an IGW attached to the VPC — traffic still won't flow unless the route table says "send internet traffic to the IGW." This route is the bridge between your subnet and the internet. The local route handles internal VPC traffic automatically.

Prerequisites

- Tasks 1–4 complete
- Internet Gateway my-igw attached to my-vpc





- **Steps**

  Step 1 — Create Public Route Table
- Go to **VPC Console → Route Tables → Create Route Table**
- Fill in:
- Name: public-rt
- VPC: my-vpc

  Click **Create Route Table**

  Screenshot: public-rt created, showing your VPC ID
- **Step 2 — Associate Both Public Subnets**
- Select public-rt → click **Subnet Associations** tab
- Click **Edit subnet associations**
- Check both:
- ✅ Public-Sub-1a
- ✅ Public-Sub-1b

  Click **Save associations**
- 📸 Screenshot: Subnet Associations tab showing both public subnets listed under public-rt

  Step 3 — Verify
- Click the **Routes** tab on public-rt
- Right now you should see only **1 route** (local):
- 10.0.0.0/16 → local
- This is expected — you'll add the internet route in Task 6

Validation

- public-rt is visible in Route Tables list
- Both Public-Sub-1a and Public-Sub-1b appear under Subnet Associations
- Private subnets are **not** associated with public-rt
- Routes tab shows only 10.0.0.0/16 → local for now (internet route comes in Task 6)

Common Errors & Fixes

Error	Cause	Fix
Subnet already associated with main RT	Default behavior — subnets auto-associate with main RT	That's fine, explicit association to public-rt overrides it
Can't find public subnets in the list	Subnet belongs to different VPC	Make sure you're filtering subnets by my-vpc
Route table not showing in dropdown	Wrong region	Confirm you're in north-1a

TASK 6 — Add Internet Route to Public Route Table

Outcome

The public route table now has a live internet route. Any EC2 launched in Public-Sub-1a or Public-Sub-1b can send and receive internet traffic through the Internet Gateway. Your full public networking path is complete:

Objective

Add a default route 0.0.0.0/0 → Internet Gateway to public-rt so that instances in public subnets can send and receive traffic to/from the internet.

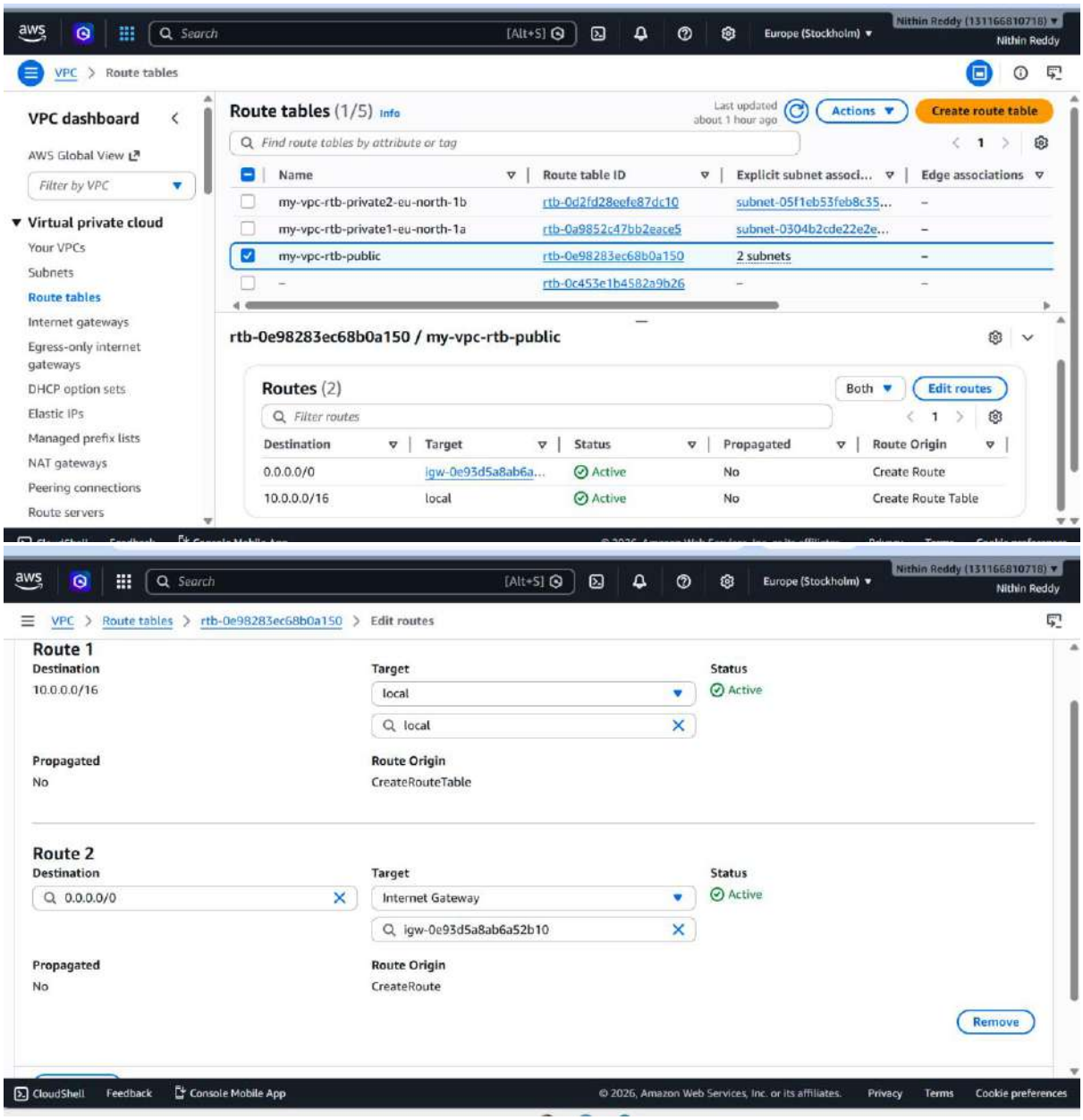
Why This Task?

This single route is what makes a subnet truly **"public."** Without it, even if your EC2 has a public IP and the IGW is attached to the VPC — packets have nowhere to go. The route tells AWS: *"for any destination outside the VPC, send traffic through the Internet Gateway."* The local route handles internal VPC traffic automatically.

Prerequisites

- Task 5 complete — public-rt exists with both public subnets associated

- Internet Gateway my-igw is attached to my-vpc



Steps

- **Step 1 — Add the Internet Route**
- VPC Console → **Route Tables** → select public-rt
- Click the **Routes** tab
- Click **Edit routes → Add route**
- Fill in:
- Destination: 0.0.0.0/0

- Target: click the dropdown → select **Internet Gateway** → choose my-igw
- Click **Save changes**
- 📸 Screenshot: Routes tab now showing **2 routes**:
- 10.0.0.0/16 → local
- 0.0.0.0/0 → igw-XXXXX
- **Step 2 — Confirm Route is Active**
- Both routes should show **State = active**

Screenshot: Both routes with green active status

Validation

- public-rt Routes tab shows exactly **2 routes**
- 0.0.0.0/0 route target is your IGW ID (starts with igw-)
- Both routes show State = **active**
- private-rt still has **only 1 route** (local) — confirm nothing was accidentally added there
- Quick test: launch an EC2 in Public-Sub-1a → it should get a public IP and be able to ping 8.8.8.8

Common Errors & Fixes

Error	Cause	Fix
Route already exists error	0.0.0.0/0 already added to this RT	Click edit routes and update the existing one instead of adding

Error	Cause	Fix
IGW not showing in Target dropdown	IGW not attached to VPC	Go to Internet Gateways → attach to my-vpc first
Route added but State = blackhole	IGW was deleted or detached	Re-attach IGW to VPC, route state will recover
Added route to private-rt by mistake	Wrong RT selected	Delete the 0.0.0.0/0 route from private-rt immediately

TASK 7 — Launch EC2 in Public Subnet + Install PHP

Outcome

A public-facing EC2 is running in your VPC's public subnet with PHP installed. HTTP and SSH are accessible from the internet, confirming the full public network path works correctly end-to-end.

Objective

Launch a t2.micro Amazon Linux 2 EC2 in Public-Sub-1a, SSH into it, and install PHP — confirming both SSH access and outbound internet work correctly.

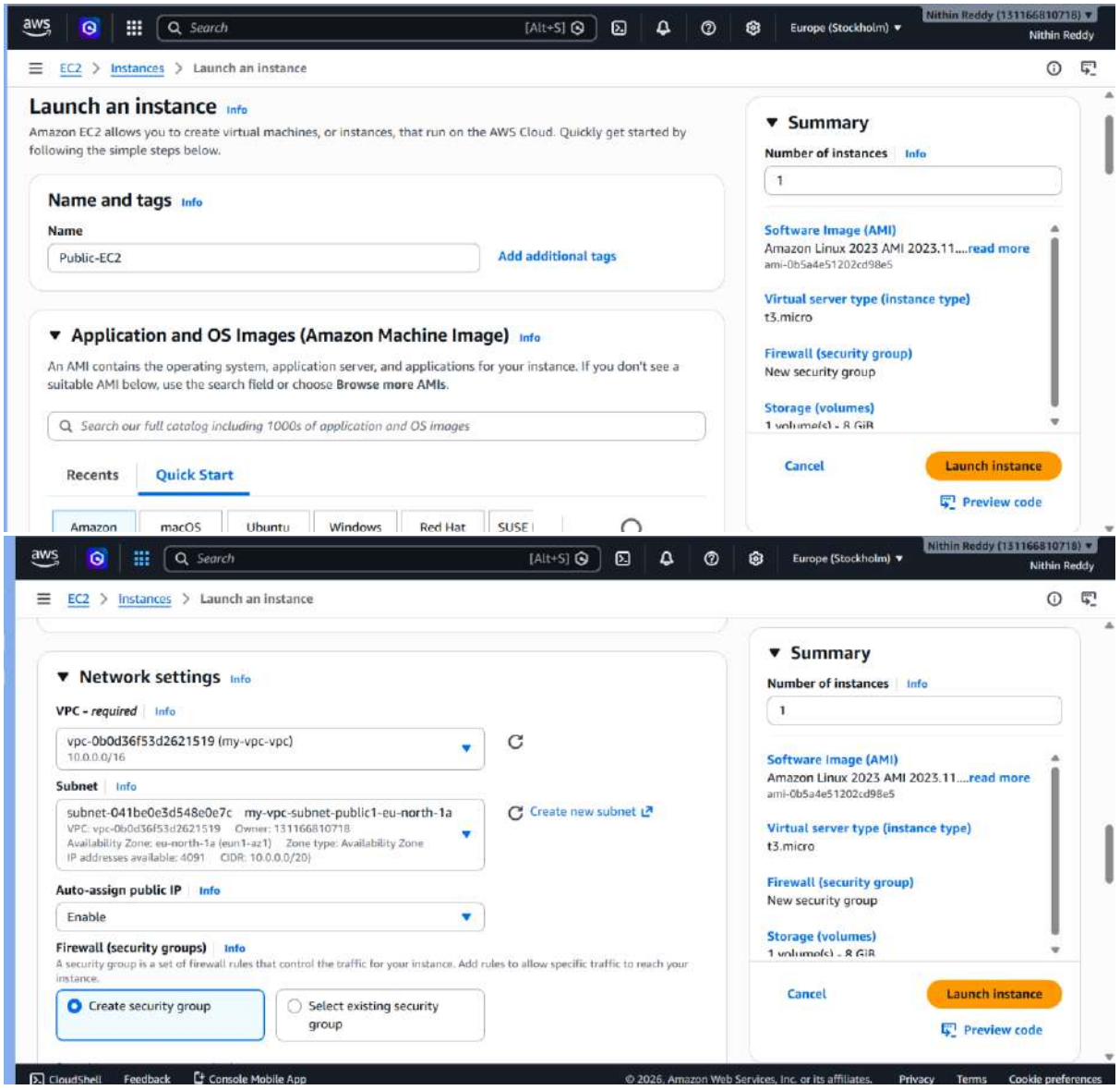
Why This Task?

This is the first real workload in your VPC. It validates everything from Tasks 1–6 end-to-end: public IP assignment, IGW routing, DNS resolution. Installing PHP proves the instance has outbound

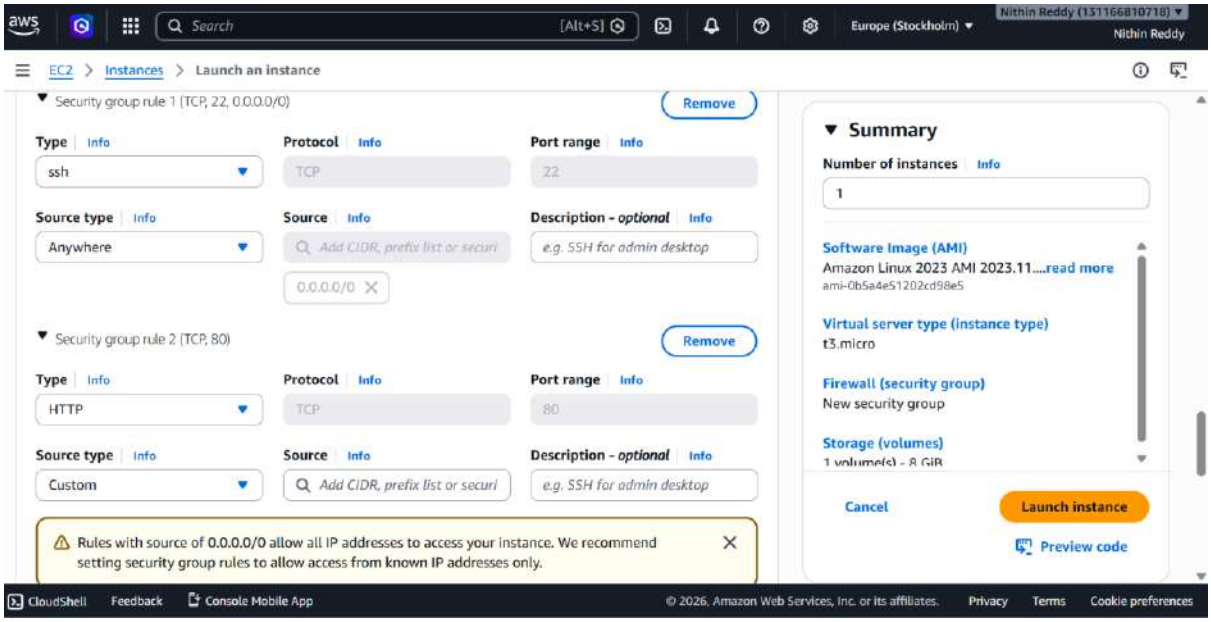
internet to download packages. It also represents the web tier in a real LAMP stack architecture.

Prerequisites

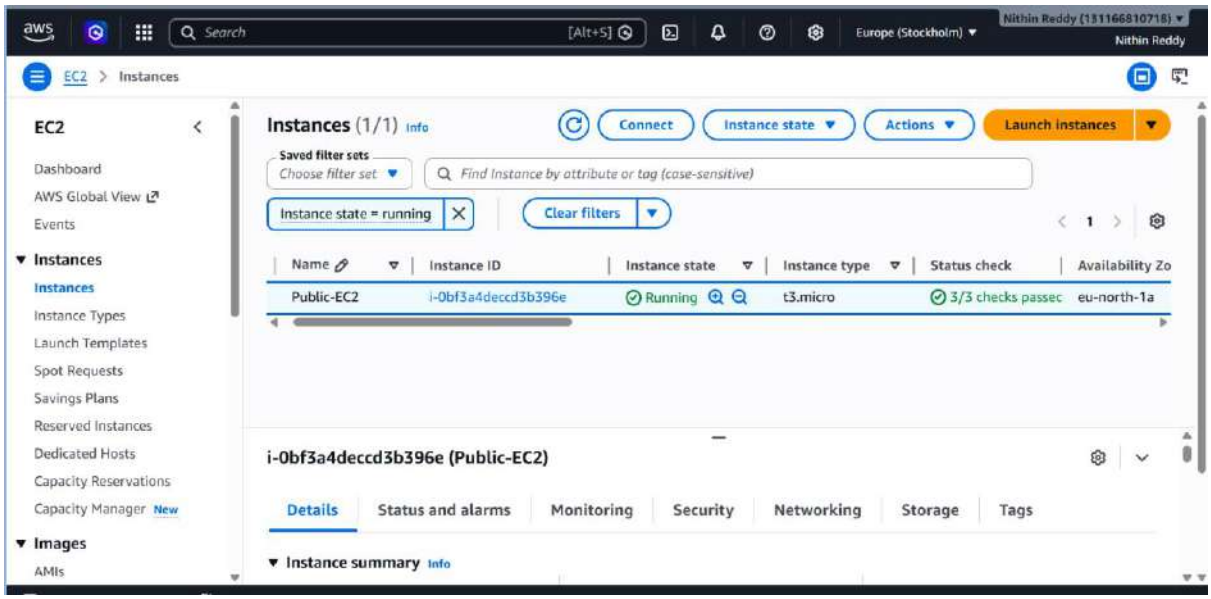
- Tasks 1–6 complete
- An EC2 Key Pair (.pem file) already created in ap-south-1
- Port 22 and 80 access available


Search [Alt+S]
Europe (Stockholm)
Nithin Reddy (131166810718)
Nithin Reddy
EC2 > Instances > Launch an instance
Launch an instance Info
Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.
Name and tags Info
Name
Public-EC2
Add additional tags
Application and OS Images (Amazon Machine Image) Info
An AMI contains the operating system, application server, and applications for your instance. If you don't see a suitable AMI below, use the search field or choose Browse more AMIs.
Search our full catalog including 1000s of application and OS images
Recents
Quick Start
Amazon
macOS
Ubuntu
Windows
Red Hat
SUSE
Summary
Number of instances Info
1
Software Image (AMI)
Amazon Linux 2023 AMI 2023.11....read more
ami-0b5a4e51202cd98e5
Virtual server type (instance type)
t3.micro
Firewall (security group)
New security group
Storage (volumes)
Cancel
Launch instance
Preview code

Search [Alt+S]
Europe (Stockholm)
Nithin Reddy (131166810718)
Nithin Reddy
EC2 > Instances > Launch an instance
Network settings Info
VPC - required Info
vpc-0b0d36f53d2621519 (my-vpc-vpc)
10.0.0.0/16
Subnet Info
subnet-041be0e3d548e0e7c my-vpc-subnet-public1-eu-north-1a
VPC: vpc-0b0d36f53d2621519 Owner: 131166810718
Availability Zone: eu-north-1a (eun1-az1) Zone type: Availability Zone
IP addresses available: 4091 CIDR: 10.0.0.0/20)
Create new subnet
Auto-assign public IP Info
Enable
Firewall (security groups) Info
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.
Create security group
Select existing security group
Summary
Number of instances Info
1
Software Image (AMI)
Amazon Linux 2023 AMI 2023.11....read more
ami-0b5a4e51202cd98e5
Virtual server type (instance type)
t3.micro
Firewall (security group)
New security group
Storage (volumes)
Cancel
Launch instance
Preview code
CloudShell
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences

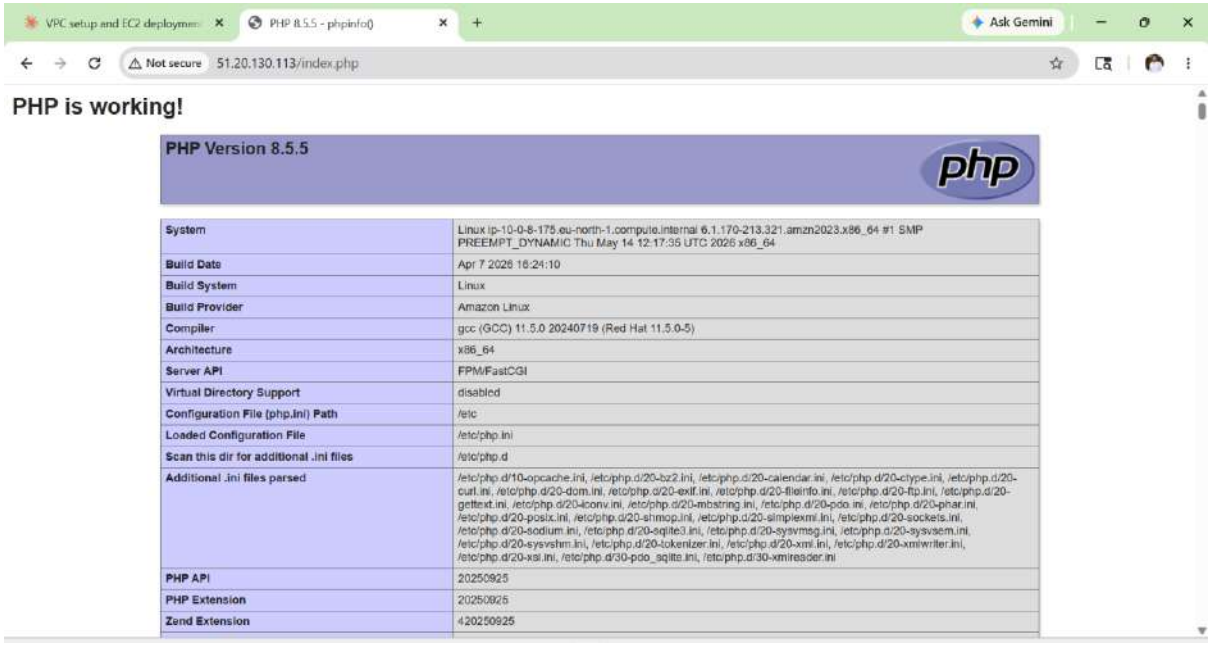


Instance launched successfully



```
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-10-0-8-175 ~]$ sudo su
[root@ip-10-0-8-175 ec2-user]# sudo yum update -y
Amazon Linux 2023 Kernel Livepatch repository                    330 kB/s |  38 kB     00:00
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-10-0-8-175 ec2-user]# sudo amazon-linux-extras enable php8.0
sudo: amazon-linux-extras: command not found
[root@ip-10-0-8-175 ec2-user]# sudo dnf install -y php php-cli php-fpm
Last metadata expiration check: 0:00:59 ago on Tue May 26 10:34:25 2026.
Dependencies resolved.
================================================================================
 Package                 Architecture    Version                  Repository  Size
================================================================================
Installing:
 php8.5                  x86_64          8.5.5-1.amzn2023.0.1     amazonlinux 14 k
 php8.5-fpm              x86_64          8.5.5-1.amzn2023.0.1     amazonlinux 3.0 M
Installing dependencies:
 apr                     x86_64          1.7.5-1.amzn2023.0.4     amazonlinux 129 k
 apr-util                x86_64          1.6.3-1.amzn2023.0.2     amazonlinux 97 k
 apr-util-lmdb           x86_64          1.6.3-1.amzn2023.0.2     amazonlinux 13 k
 generic-logos-httpd     noarch          18.0.0-12.amzn2023.0.3   amazonlinux 19 k
 httpd-core              x86_64          2.4.66-1.amzn2023.0.1    amazonlinux 1.4 M
 httpd-filesystem        noarch          2.4.66-1.amzn2023.0.1    amazonlinux 13 k
 httpd-tools             x86_64          2.4.66-1.amzn2023.0.1    amazonlinux 81 k
 libbrotli               x86_64          1.0.9-4.amzn2023.0.2     amazonlinux 315 k
 libsodium               x86_64          1.0.19-5.amzn2023        amazonlinux 174 k
 libxslt                 x86_64          1.1.43-1.amzn2023.0.3    amazonlinux 183 k
```

Output



Validation

- EC2 shows Public IP in console
- SSH connection succeeds
- php -v returns PHP 8.0.x
- curl http://localhost/index.php returns PHP HTML
- From browser: http://<PUBLIC-IP>/index.php loads the PHP page
- Instance passes 2/2 status checks

Common Errors & Fixes

Error	Cause	Fix
Connection timed out on SSH	Port 22 blocked in SG	Add SSH inbound rule to security group
Permission denied (publickey)	Wrong key or wrong user	Use ec2-user, ensure correct .pem
EC2 has no public IP	Auto-assign was off	Terminate, fix subnet setting (Task 3), relaunch
curl returns nothing	Apache not started	Run sudo systemctl start httpd
amazon-linux-extras not found	Using Amazon Linux 2023 instead of AL2	Use Amazon Linux 2 AMI specifically

TASK 8 Configure NAT gateway in public subnet and connect to private instance.

Outcome

Private EC2 instances can now securely download packages and make API calls via the NAT Gateway, while remaining completely unreachable from the public internet. The two-tier network (public + private) is fully operational.

Objective

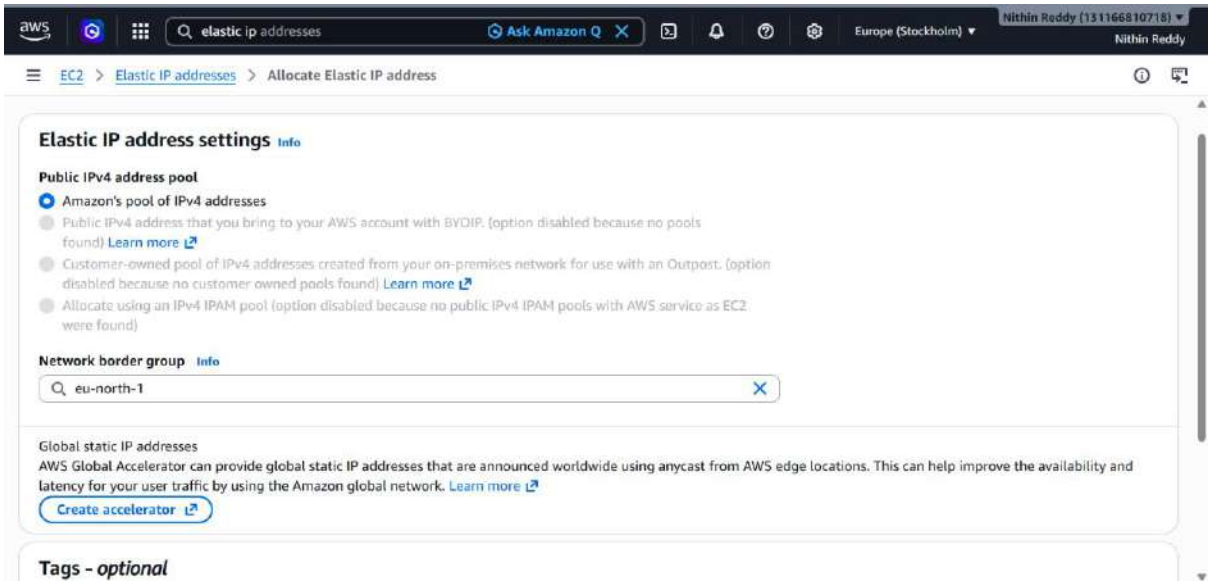
Deploy a NAT Gateway in the public subnet, add its route to the private route table, launch an EC2 in the private subnet, and verify the private instance can reach the internet through NAT.

Why This Task?

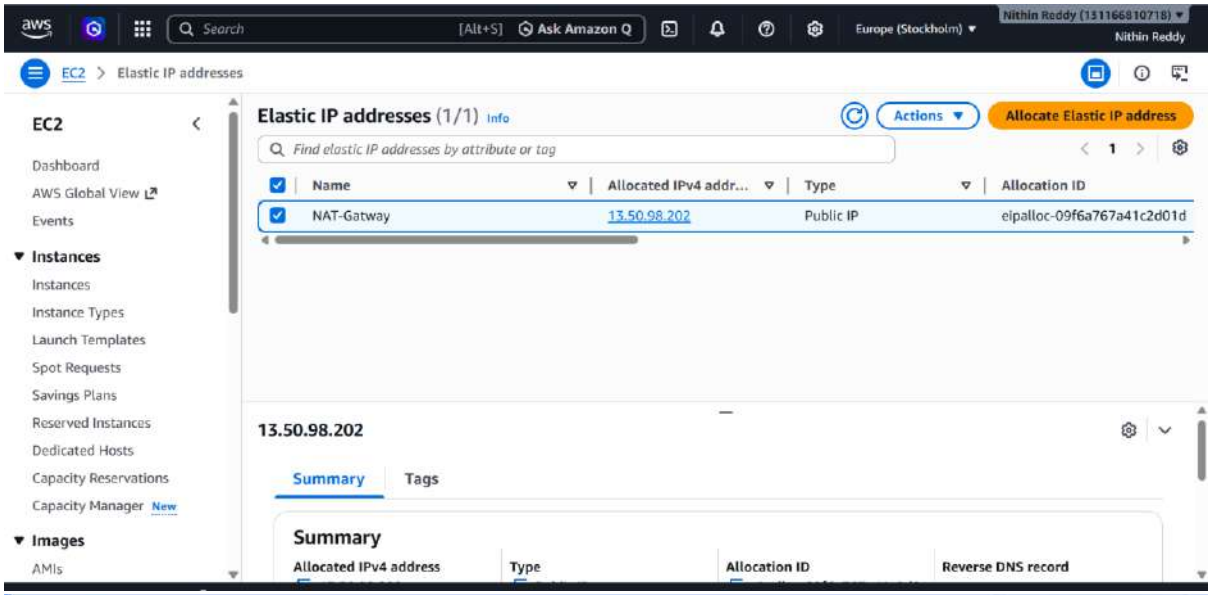
Private instances have no public IP — they can't reach the internet directly. But they still need outbound access for things like package downloads and security patches. NAT Gateway provides exactly this: **outbound-only internet access** from private subnets. Inbound traffic from the internet is still blocked, keeping your private instances secure.

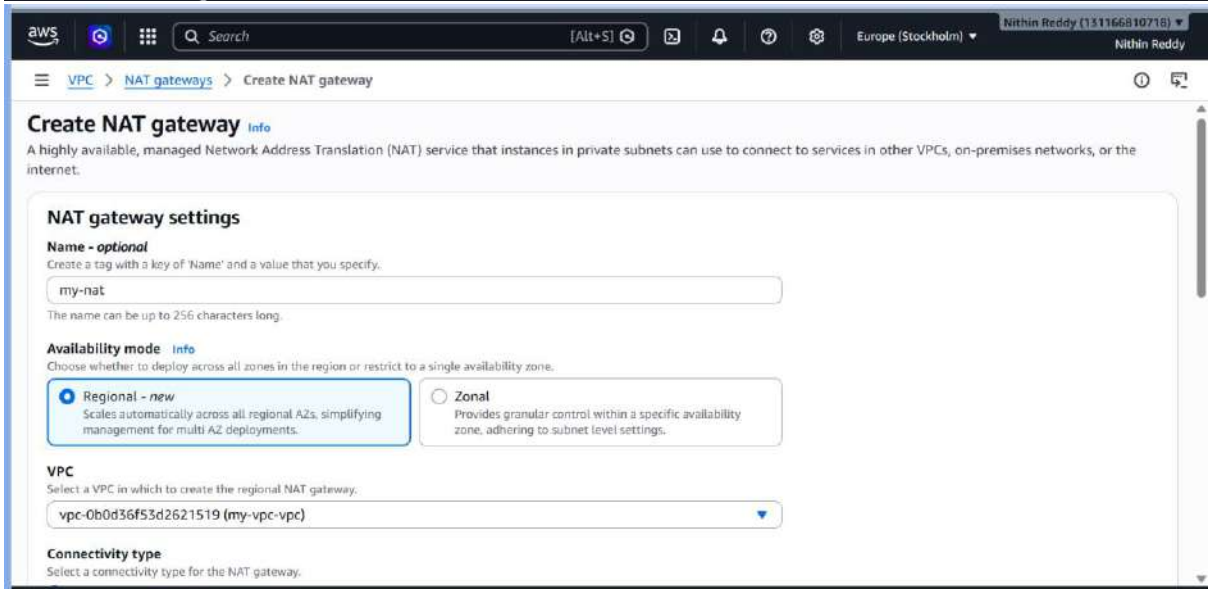
Prerequisites

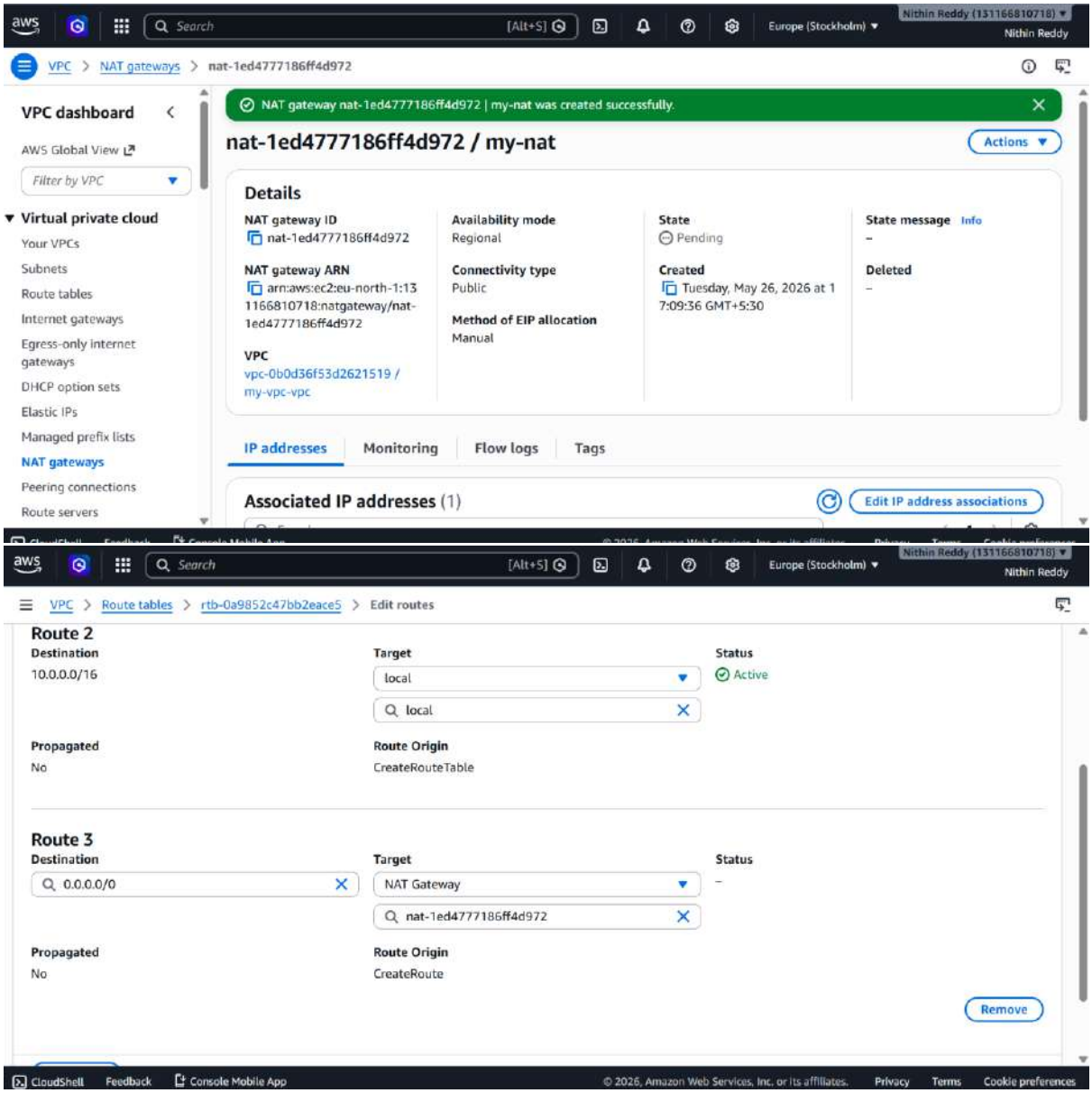
- Tasks 1–7 complete
- Elastic IP quota available (check EC2 → Elastic IPs)


elastic ip addresses
Ask Amazon Q
Europe (Stockholm)
Nithin Reddy (131166810718)
Nithin Reddy
EC2 > Elastic IP addresses > Allocate Elastic IP address
Elastic IP address settings Info
Public IPv4 address pool
Amazon's pool of IPv4 addresses
Public IPv4 address that you bring to your AWS account with BYOIP. (option disabled because no pools found) Learn more
Customer-owned pool of IPv4 addresses created from your on-premises network for use with an Outpost. (option disabled because no customer owned pools found) Learn more
Allocate using an IPv4 IPAM pool (option disabled because no public IPv4 IPAM pools with AWS service as EC2 were found)
Network border group Info
eu-north-1
Global static IP addresses
AWS Global Accelerator can provide global static IP addresses that are announced worldwide using anycast from AWS edge locations. This can help improve the availability and latency for your user traffic by using the Amazon global network. Learn more
Create accelerator
Tags - optional

Created IP


Search
[Alt+S]
Ask Amazon Q
Europe (Stockholm)
Nithin Reddy (131166810718)
Nithin Reddy
EC2 > Elastic IP addresses
EC2
Dashboard
AWS Global View
Events
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations
Capacity Manager New
Images
AMIs
Elastic IP addresses (1/1) Info
Actions
Allocate Elastic IP address
Find elastic IP addresses by attribute or tag
Name
Allocated IPv4 addr...
Type
Allocation ID
NAT-Gatway
13.50.98.202
Public IP
eipalloc-09f6a767a41c2d01d
13.50.98.202
Summary
Tags
Summary
Allocated IPv4 address
Type
Allocation ID
Reverse DNS record


Search
[Alt+S]
Europe (Stockholm)
Nithin Reddy (131166810718)
Nithin Reddy
VPC > NAT gateways > Create NAT gateway
Create NAT gateway Info
A highly available, managed Network Address Translation (NAT) service that instances in private subnets can use to connect to services in other VPCs, on-premises networks, or the internet.
NAT gateway settings
Name - optional
Create a tag with a key of 'Name' and a value that you specify.
my-nat
The name can be up to 256 characters long.
Availability mode Info
Choose whether to deploy across all zones in the region or restrict to a single availability zone.
Regional - new
Scales automatically across all regional AZs, simplifying management for multi AZ deployments.
Zonal
Provides granular control within a specific availability zone, adhering to subnet level settings.
VPC
Select a VPC in which to create the regional NAT gateway.
vpc-0b0d36f53d2621519 (my-vpc-vpc)
Connectivity type
Select a connectivity type for the NAT gateway.



Steps

PART A — Create NAT Gateway (AWS Console)

Step 1 — Allocate Elastic IP

1. EC2 Console → **Elastic IPs → Allocate Elastic IP address**
2. Network border group: keep default
3. Click **Allocate**
4. 📸 Note down the Elastic IP address

Step 2 — Create NAT Gateway

1. VPC Console → **NAT Gateways → Create NAT Gateway**
2. Fill in:
    - Name: my-nat
    - Subnet: Public-Sub-1b ← must be a PUBLIC subnet
    - Connectivity type: **Public**
    - Elastic IP: select the one you just allocated
3. Click **Create NAT Gateway**
4. ⏳ Wait **2–3 minutes** until Status = **Available**
5. 📸 Screenshot: NAT Gateway Status = Available

PART B — Update Private Route Table

Step 3 — Add NAT Route to private-rt

1. VPC Console → **Route Tables** → select private-rt
2. **Routes tab → Edit routes → Add route**
3. Fill in:
    - Destination: 0.0.0.0/0
    - Target: **NAT Gateway** → select my-nat
4. Save changes
5. 📸 Screenshot: private-rt now showing 2 routes:
    - 10.0.0.0/16 → local
    - 0.0.0.0/0 → nat-XXXXX

PART C — Launch Private EC2

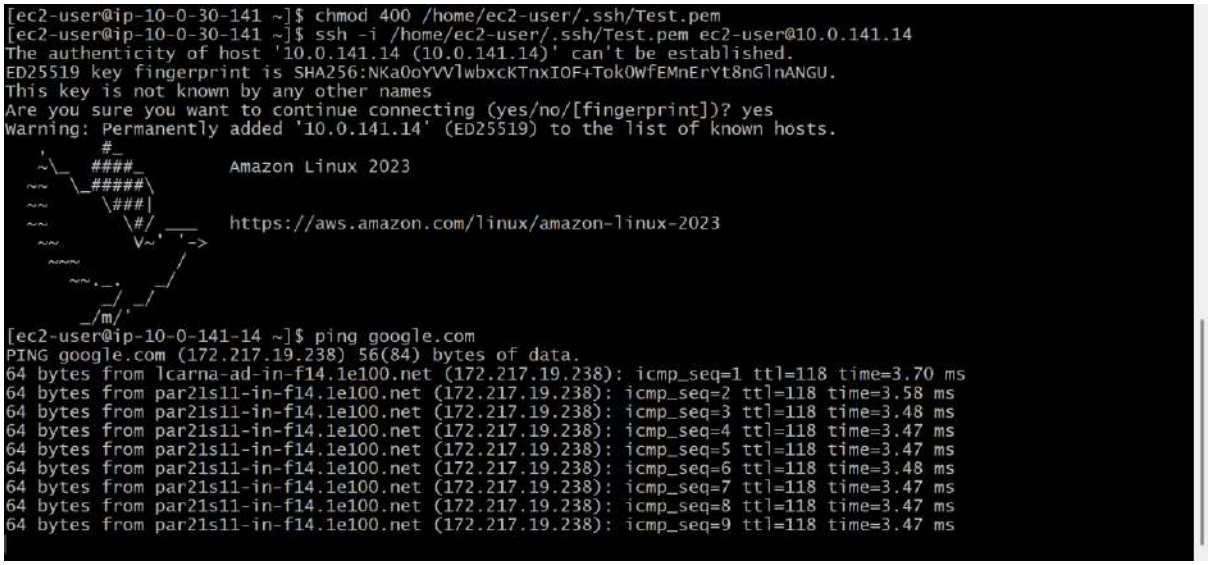
Step 4 — Create Private Security Group

1. EC2 → **Security Groups → Create Security Group**
2. Fill in:
    - Name: private-sg
    - VPC: my-vpc
3. Inbound rule:
    - Type: SSH, Port: 22, Source: 10.0.0.0/16
4. Create

Step 5 — Launch Private EC2

1. EC2 → **Launch Instance**
2. Fill in:
    - Name: private-ec2
    - AMI: **Amazon Linux 2023**
    - Type: t2.micro
    - Key pair: **same .pem as before**
3. Network settings:
    - VPC: my-vpc
    - Subnet: Private-Sub-1a
    - Auto-assign public IP: **Disable**
    - Security Group: private-sg
4. Launch
5. 📸 Screenshot: private-ec2 running with **no Public IP** in console

Final output :



Validation

- NAT Gateway Status = **Available**
- private-rt has 2 routes: local + 0.0.0.0/0 → NAT
- private-ec2 has **no Public IP** in console
- SSH from public EC2 → private EC2 succeeds
- curl https://checkip.amazonaws.com returns NAT Gateway's Elastic IP
- sudo dnf update -y completes successfully on private EC2

Common Errors & Fixes

Error	Cause	Fix
SSH to private EC2 times out	Security group source wrong	Confirm `private-sg` allows SSH from `10.0.0.0/16`
`curl checkip` hangs	NAT route not in private-rt	Check `private-rt` has `0.0.0.0/0 → my-nat`
`curl checkip` returns private IP	Route missing or wrong target	Delete and re-add the NAT route in `private-rt`
`Permission denied (publickey)`	Key not copied or wrong permissions	Run `chmod 400 ~/.ssh/Test.pem` on public EC2

Error	Cause	Fix
NAT Gateway stuck in Pending	Normal — takes time	Wait 3–5 mins and refresh
`scp` fails locally	Wrong path to Test.pem	Run `scp` from the folder where Test.pem exists

-

TASK 9 — Install Apache Tomcat on Private EC2 + Deploy Sample App

Objective

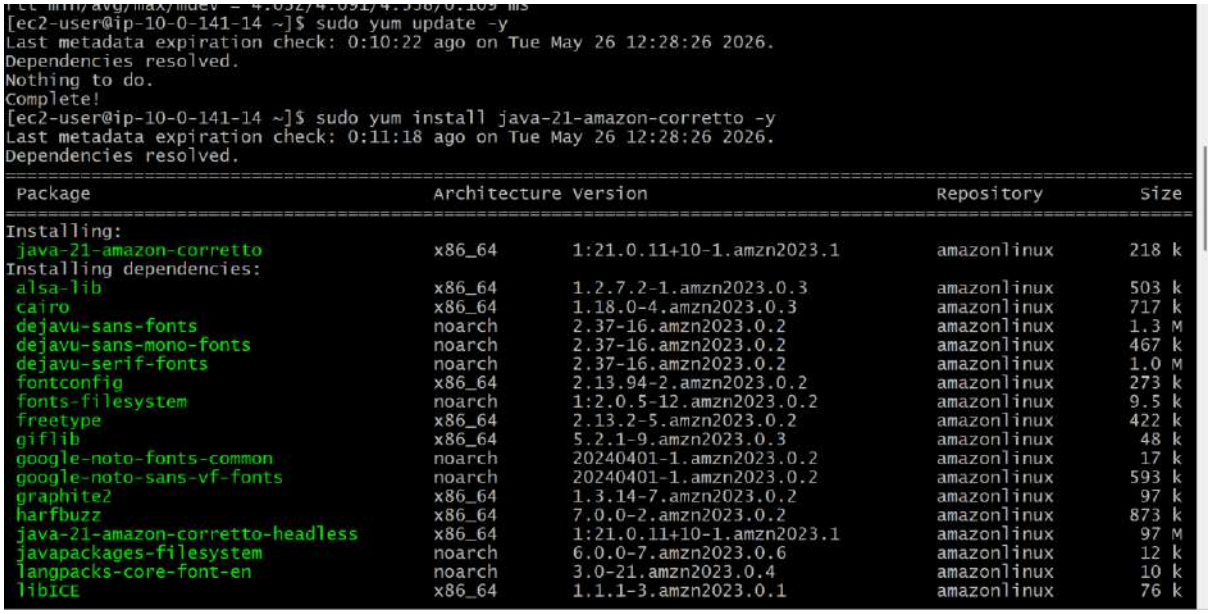
Install Java 11 and Apache Tomcat 10 on the private EC2, start Tomcat as a service, and verify the built-in sample application is deployed and running on port 8080.

Why This Task?

This simulates a real-world Java application server running in a private subnet — a standard pattern in enterprise architectures. The private EC2 runs your application backend, unreachable directly from the internet, while the public EC2 would act as a reverse proxy or load balancer in front of it. NAT Gateway (Task 8) enables Tomcat to be downloaded even though the instance has no public IP.

Prerequisites

- Task 8 complete
- SSH access to private EC2 via jump host
- NAT Gateway working (yum update succeeds on private EC2)



```
[ec2-user@ip-10-0-141-14 ~]$ sudo yum update -y
Last metadata expiration check: 0:10:22 ago on Tue May 26 12:28:26 2026.
Dependencies resolved.
Nothing to do.
Complete!
[ec2-user@ip-10-0-141-14 ~]$ sudo yum install java-21-amazon-corretto -y
Last metadata expiration check: 0:11:18 ago on Tue May 26 12:28:26 2026.
Dependencies resolved.
```

Package	Architecture	Version	Repository	Size
Installing:				
java-21-amazon-corretto	x86_64	1:21.0.11+10-1.amzn2023.1	amazonlinux	218 k
Installing dependencies:				
alsa-lib	x86_64	1.2.7.2-1.amzn2023.0.3	amazonlinux	503 k
cairo	x86_64	1.18.0-4.amzn2023.0.3	amazonlinux	717 k
dejavu-sans-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	1.3 M
dejavu-sans-mono-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	467 k
dejavu-serif-fonts	noarch	2.37-16.amzn2023.0.2	amazonlinux	1.0 M
fontconfig	x86_64	2.13.94-2.amzn2023.0.2	amazonlinux	273 k
fonts-filesystem	noarch	1:2.0.5-12.amzn2023.0.2	amazonlinux	9.5 k
freetype	x86_64	2.13.2-5.amzn2023.0.2	amazonlinux	422 k
giflib	x86_64	5.2.1-9.amzn2023.0.3	amazonlinux	48 k
google-noto-fonts-common	noarch	20240401-1.amzn2023.0.2	amazonlinux	17 k
google-noto-sans-vf-fonts	noarch	20240401-1.amzn2023.0.2	amazonlinux	593 k
graphite2	x86_64	1.3.14-7.amzn2023.0.2	amazonlinux	97 k
harfbuzz	x86_64	7.0.0-2.amzn2023.0.2	amazonlinux	873 k
java-21-amazon-corretto-headless	x86_64	1:21.0.11+10-1.amzn2023.1	amazonlinux	97 M
javapackages-filesystem	noarch	6.0.0-7.amzn2023.0.6	amazonlinux	12 k
langpacks-core-font-en	noarch	3.0-21.amzn2023.0.4	amazonlinux	10 k
libICE	x86_64	1.1.1-3.amzn2023.0.1	amazonlinux	76 k

Steps

1 — SSH into Private EC2 via Bastion (SSH Tunneling)

2 — Install Java (Tomcat requires Java

3 — Download & Install Apache Tomcat

4 — Start Tomcat

5 — Configure Security Group for Port 8080

6 — Deploy a Sample WAR Application

7 — Make Tomcat Start on Reboot (Systemd Service)

```
OpenJDK 64-Bit Server VM Corretto-21.0.11.10.1 (build 21.0.11+10-LTS, mixed mode, sharing)
[ec2-user@ip-10-0-141-14 ~]$ dirname $(dirname $(readlink -f $(which java)))
/usr/lib/jvm/java-21-amazon-corretto.x86_64
[ec2-user@ip-10-0-141-14 ~]$ cd /opt
[ec2-user@ip-10-0-141-14 opt]$ sudo wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.55/bin/apache-tomcat-1
0.1.55.tar.gz
--2026-05-26 12:41:26--  https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.55/bin/apache-tomcat-10.1.55.tar.gz
Resolving dlcdn.apache.org (dlcdn.apache.org)... 151.101.2.132, 2a04:4e42::644
Connecting to dlcdn.apache.org (dlcdn.apache.org)|151.101.2.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 14343328 (14M) [application/x-gzip]
Saving to: 'apache-tomcat-10.1.55.tar.gz'

apache-tomcat-10.1.55.tar.g 100%[=====================================>]  13.68M  --.-KB/s    in 0.05s

2026-05-26 12:41:26 (269 MB/s) - 'apache-tomcat-10.1.55.tar.gz' saved [14343328/14343328]

[ec2-user@ip-10-0-141-14 opt]$ sudo wget https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.55/bin/apache-t
omcat-10.1.55.tar.gz
--2026-05-26 12:41:35--  https://archive.apache.org/dist/tomcat/tomcat-10/v10.1.55/bin/apache-tomcat-10.1.55.ta
r.gz
Resolving archive.apache.org (archive.apache.org)... 65.108.204.189, 2a01:4f9:1a:a084::2
Connecting to archive.apache.org (archive.apache.org)|65.108.204.189|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 14343328 (14M) [application/x-gzip]
Saving to: 'apache-tomcat-10.1.55.tar.gz.1'

apache-tomcat-10.1.55.tar.g 100%[=====================================>]  13.68M  8.08MB/s    in 1.7s

2026-05-26 12:41:37 (8.08 MB/s) - 'apache-tomcat-10.1.55.tar.gz.1' saved [14343328/14343328]

[ec2-user@ip-10-0-141-14 opt]$
```

Final output

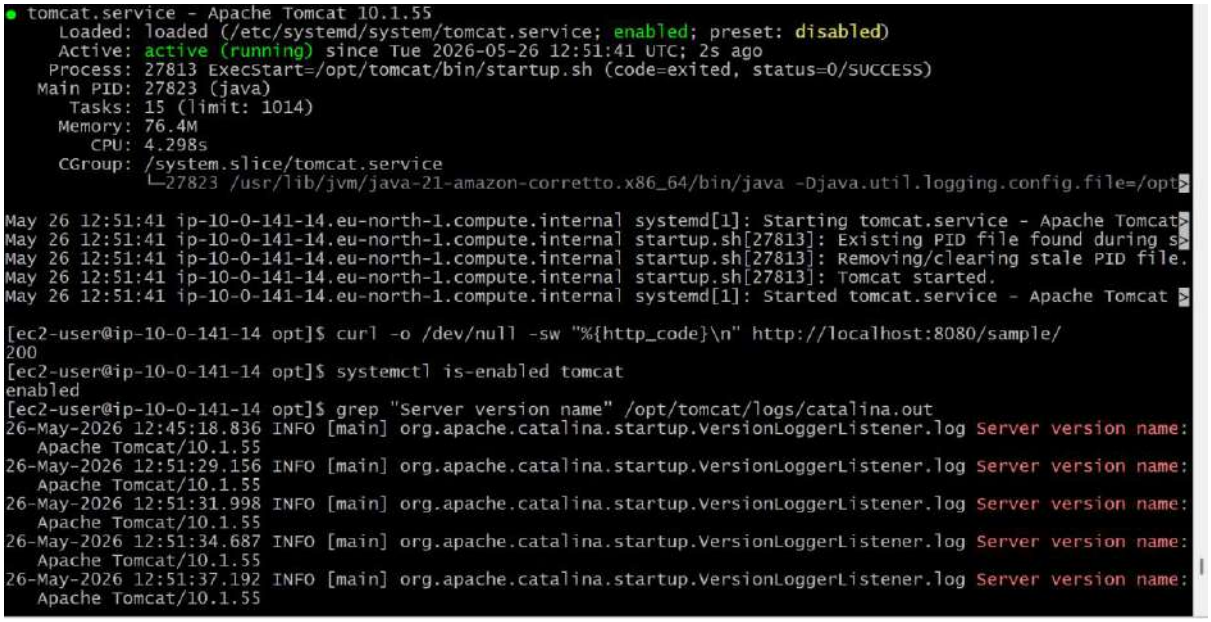
```
[Install]
WantedBy=multi-user.target
EOF
[ec2-user@ip-10-0-141-14 opt]$ cat /etc/systemd/system/tomcat.service
[Unit]
Description=Apache Tomcat 10.1.55
After=network.target

[Service]
Type=forking
Environment=JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto.x86_64
Environment=CATALINA_PID=/opt/tomcat/temp/tomcat.pid
Environment=CATALINA_HOME=/opt/tomcat
Environment=CATALINA_BASE=/opt/tomcat
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh
User=ec2-user
Restart=always
```

```
Using CATALINA_OPTS:
Tomcat started.
[ec2-user@ip-10-0-141-14 opt]$ sudo ss -tlnp | grep 8080
LISTEN 0      100                *:8080            *:*    users:(("java",pid=27248,fd=44))
[ec2-user@ip-10-0-141-14 opt]$ sudo chown -R ec2-user:ec2-user /opt/tomcat
[ec2-user@ip-10-0-141-14 opt]$ ls /opt/tomcat/webapps/
ROOT  docs  examples  host-manager  manager  sample  sample.war
[ec2-user@ip-10-0-141-14 opt]$ ls /opt/tomcat/webapps/
ROOT  docs  examples  host-manager  manager  sample  sample.war
```

- ✅ Java 21
- ✅ Tomcat 10.1.55
- ✅ Service: active + enabled
- ✅ Port 8080 listening

✅ HTTP Status: 200



What I Learned from Task 9

- Private EC2s are unreachable from internet — that's intentional security
- NAT Gateway = outbound-only internet for private subnets (download packages, no inbound)
- Bastion Host = your "jump server" to manage private resources
- Tomcat listens on 8080, deploys .war files automatically
- Security Groups act as instance-level firewalls — always explicit allow

Task 10: Configure VPC Flow Logs → Store in S3 & CloudWatch

Objective

- VPC Flow Logs capture **all IP traffic** going to/from your network interfaces in your VPC. You'll store them in **both S3** (for long-term storage/analysis) and **CloudWatch Logs** (for real-time monitoring and alerting).
- **Why this matters:** This is the foundation of cloud network security monitoring. You can detect port scans, unauthorized access attempts, traffic anomalies, and debug connectivity issues.

Task 10 — Final Outcome

What You Successfully Built

S3 Storage

- ✅ S3 bucket vpc-flow-logs-131166810718 created
- ✅ Bucket policy allowing delivery.logs.amazonaws.com to write
- ✅ Flow log files appearing under AWSLogs/ prefix as .gz files
- ✅ Long-term cheap storage for all VPC traffic records

CloudWatch Monitoring

- ✅ Log group /vpc/flow-logs created with 7-day retention
- ✅ IAM Role VPCFlowLogsRole with correct trust policy
- ✅ Real-time log streams per ENI visible
- ✅ Every ACCEPT and REJECT record captured live

Flow Logs

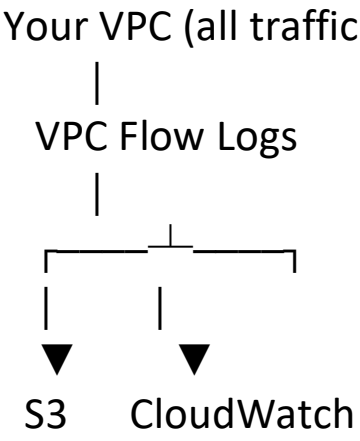
- ✅ flowlog-to-s3 → Status: Active
- ✅ flowlog-to-cloudwatch → Status: Active
- ✅ Both capturing ALL traffic (Accept + Reject)

- ✅ 1-minute aggregation interval

Security Monitoring

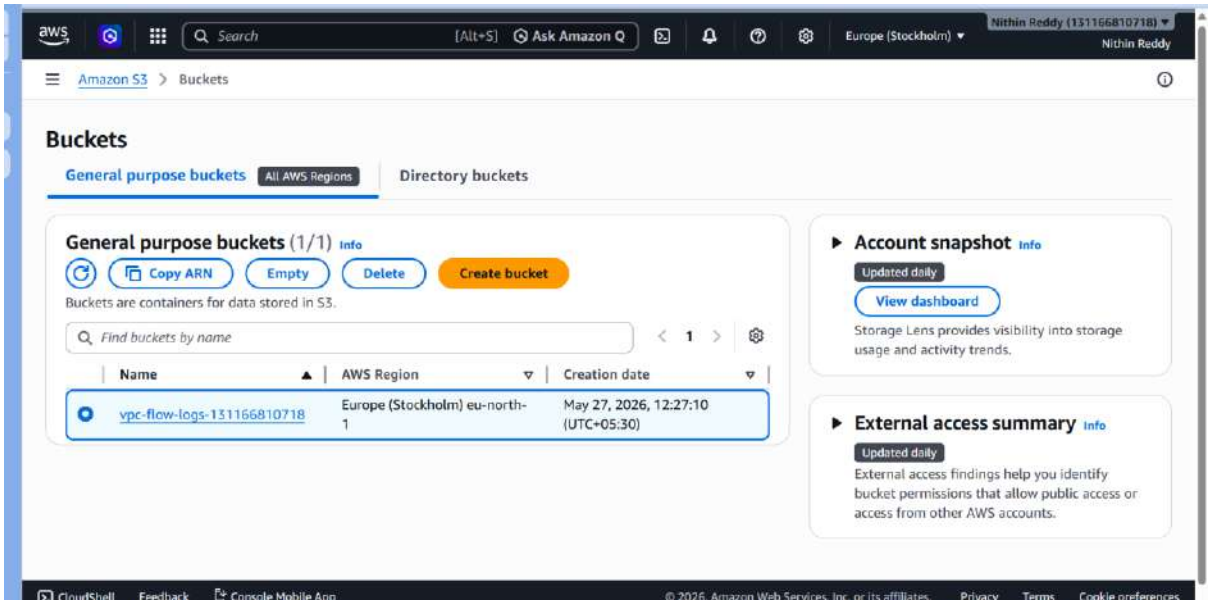
- ✅ Metric filter RejectedTraffic created
- ✅ Monitoring VPCFlowLogs/RejectedConnections
- ✅ Any blocked traffic now measurable and alertable

Architecture

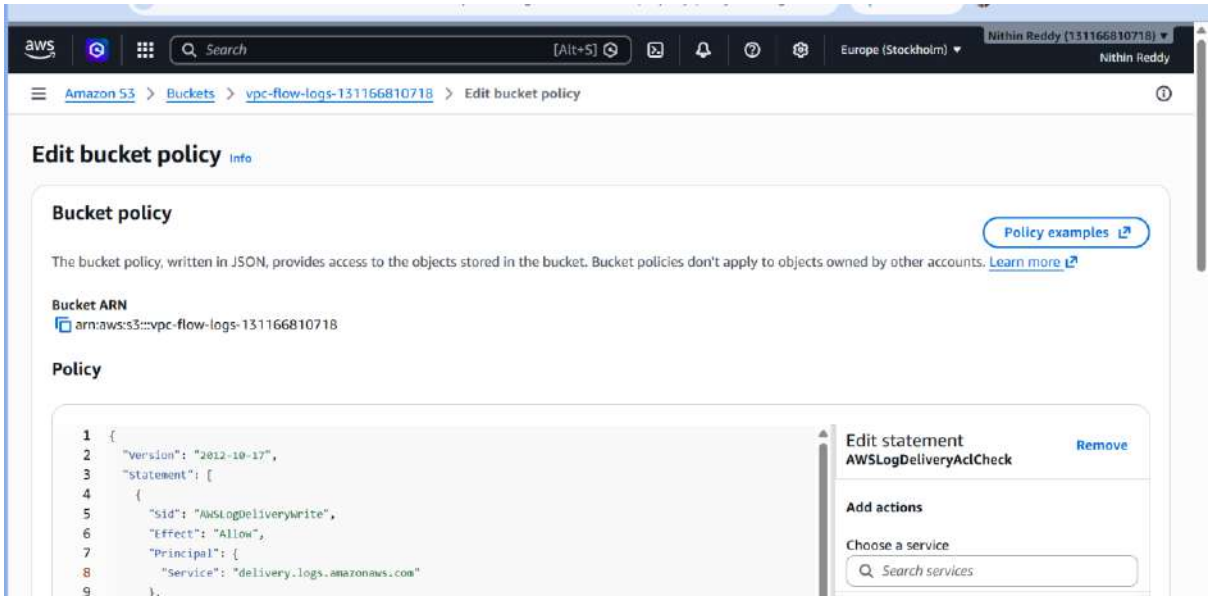


Prerequisites

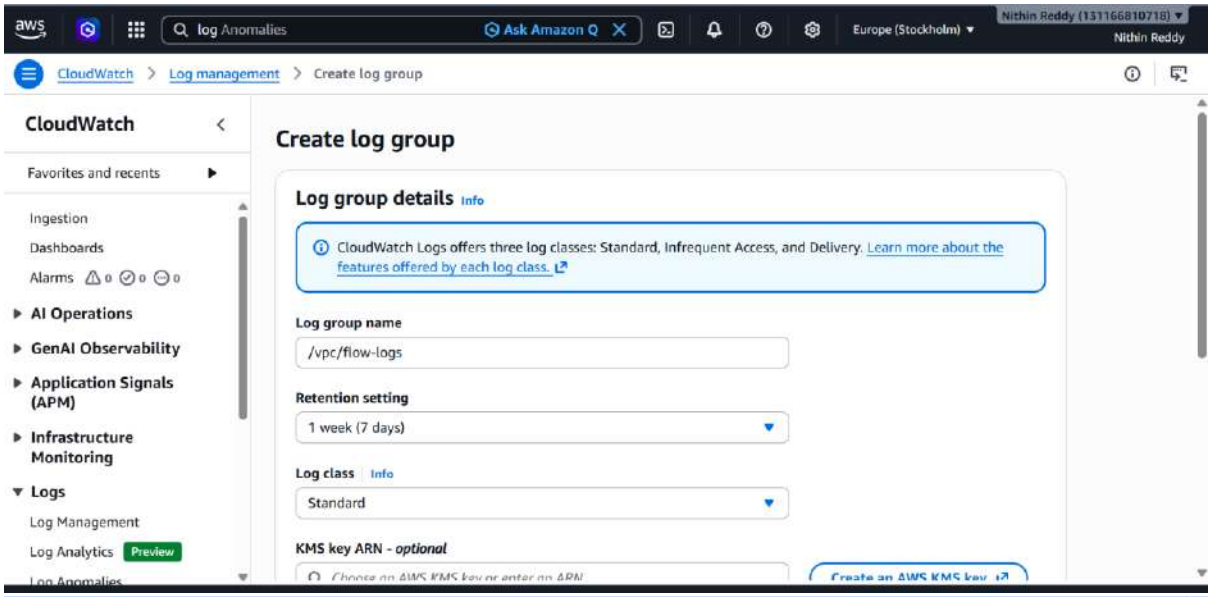
Requirement	Why
An existing VPC	Flow logs attach to a VPC/Subnet/ENI
IAM permissions	To create roles, S3 buckets, log groups
An S3 bucket (we'll create it)	Destination for flow logs
CloudWatch Log Group (we'll create it)	Destination for flow logs

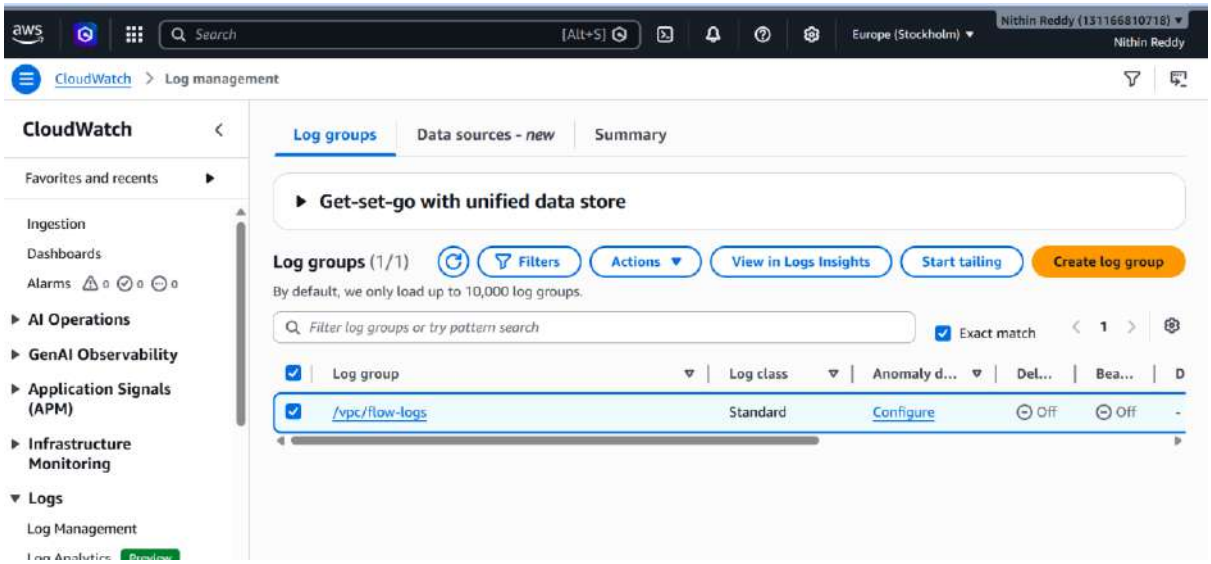


Created S3 Buckets



Created Logs

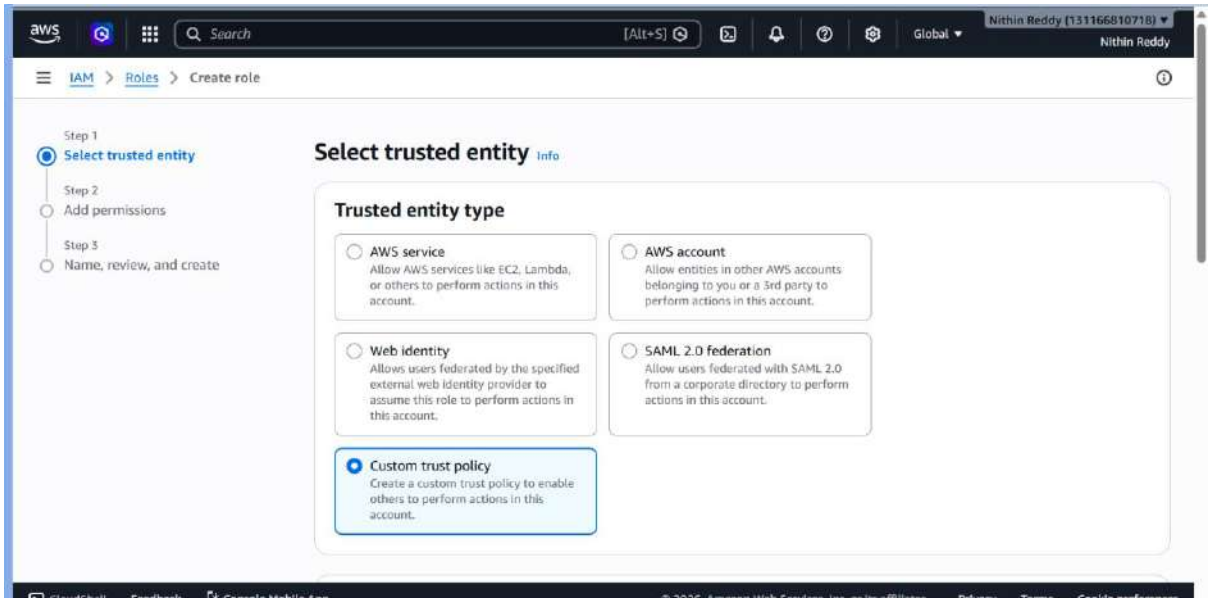




Now creating IAM Role

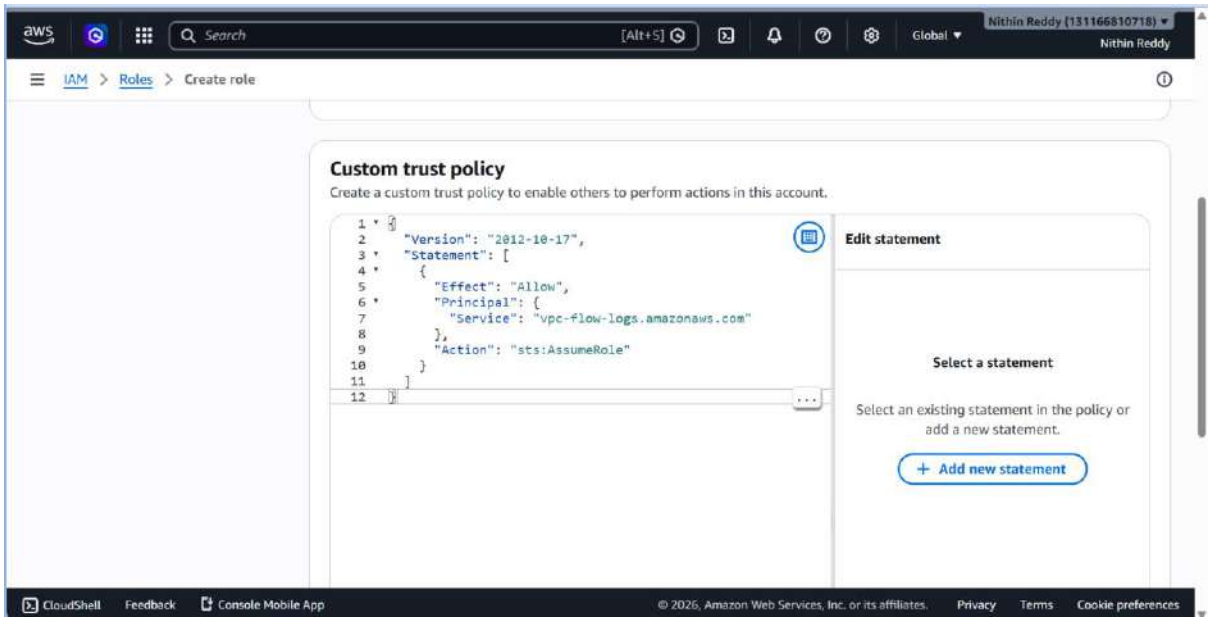
Go to IAM → Roles → Create role

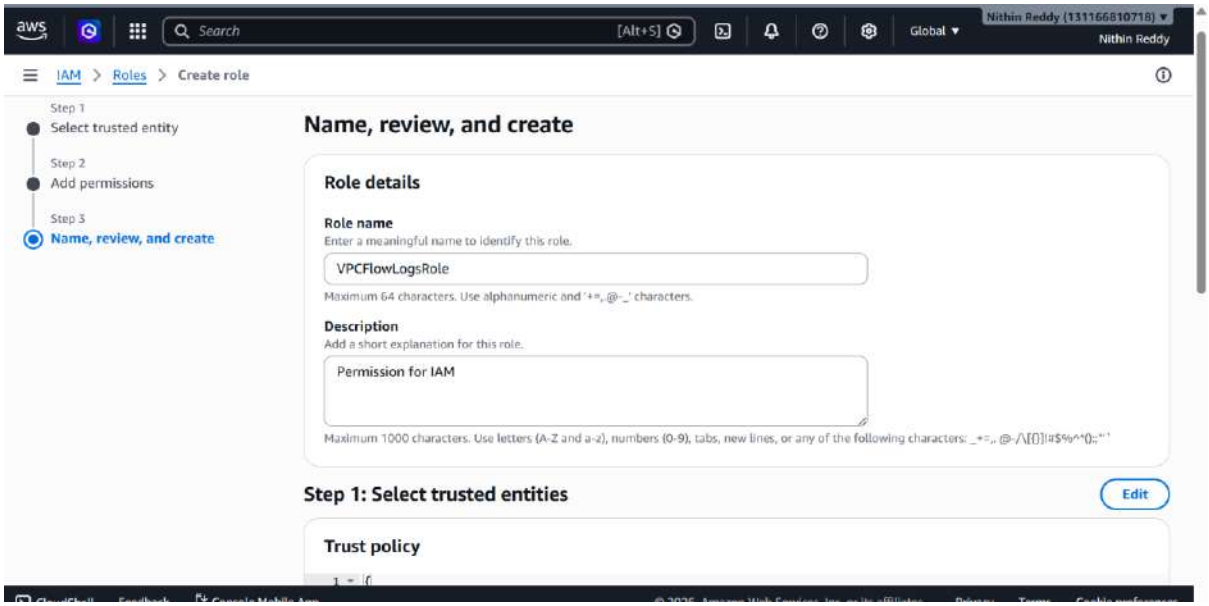
- Trusted entity type → **Custom trust policy**
- Delete the default text and paste this:



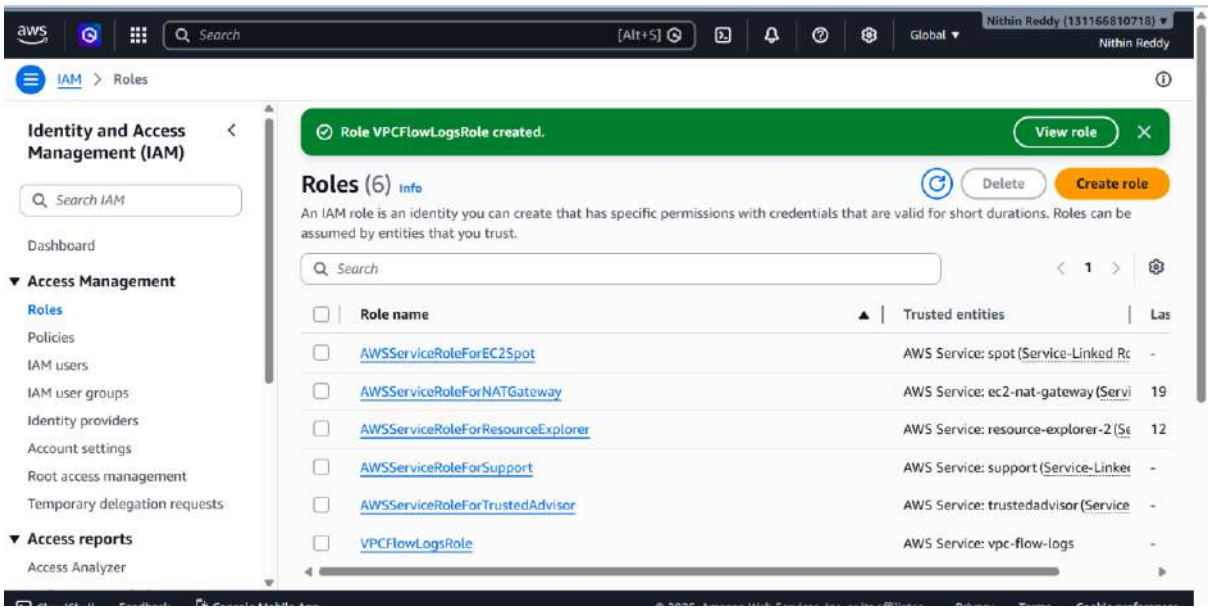
Add Permissions

- On the permissions page → click **Create inline policy** (bottom right, or skip and do after)
- Actually: **skip adding permissions here, click Next**
- Role name: VPCFlowLogsRole
- Click **Create role**





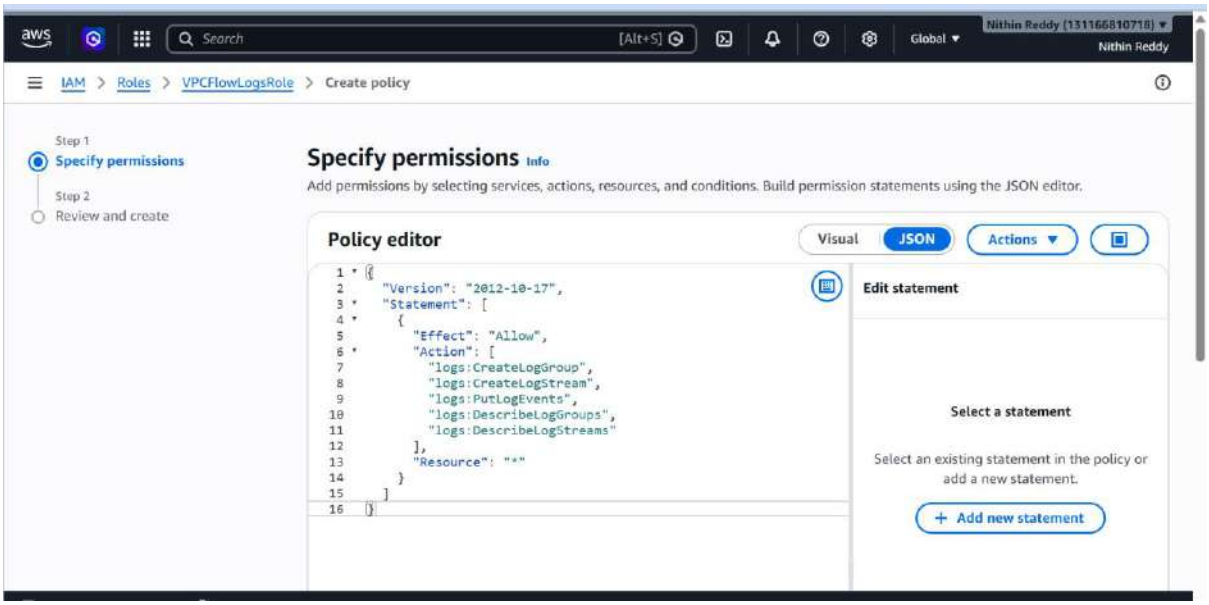
Created IAM with permissions



Add Inline Policy to the Role

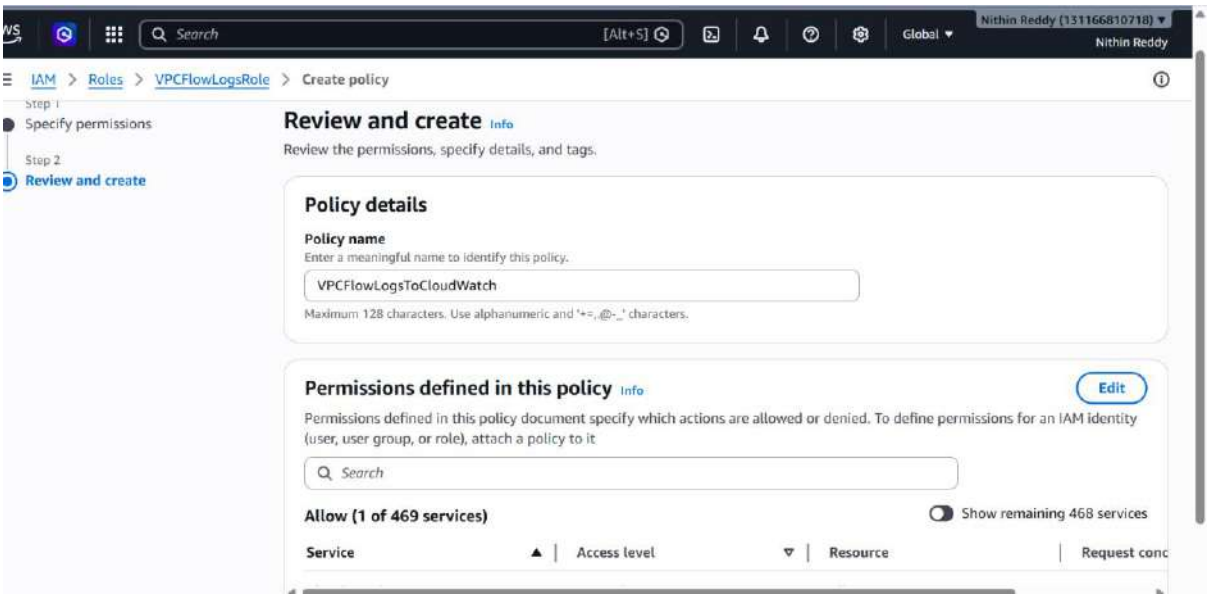
- Go to **IAM → Roles → VPCFlowLogsRole**
- Click **Add permissions → Create inline policy**

- Click **JSON tab** → delete existing text → paste:



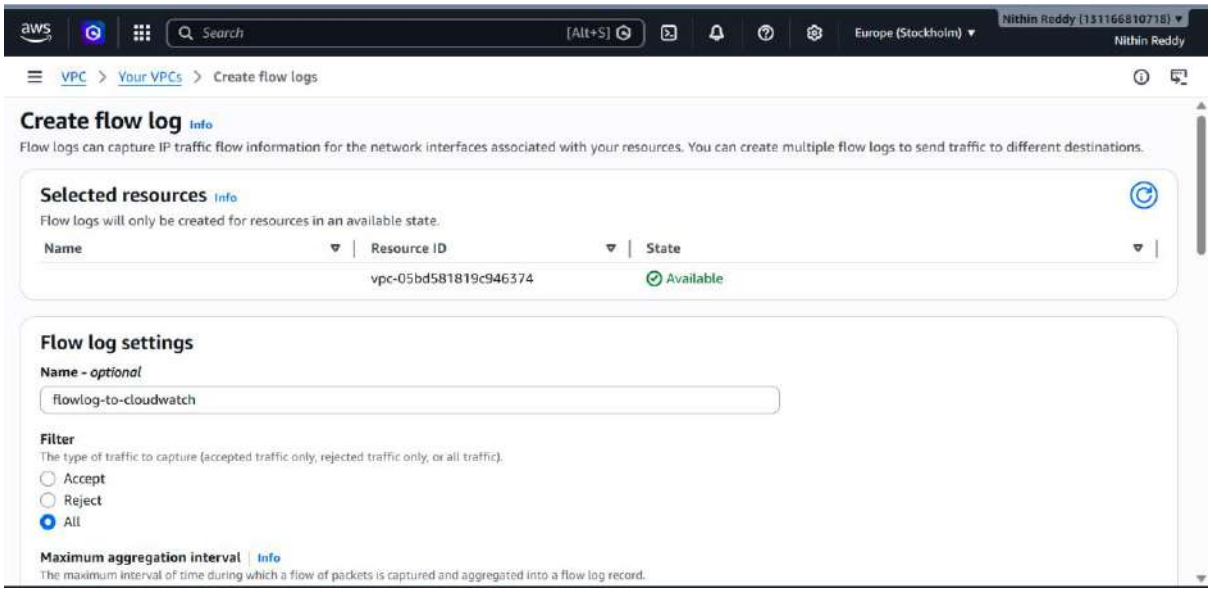
Click **Next**

- Policy name: VPCFlowLogsToCloudWatch
- Click **Create policy**
- ✅ Confirm role VPCFlowLogsRole exists with the inline policy attached.



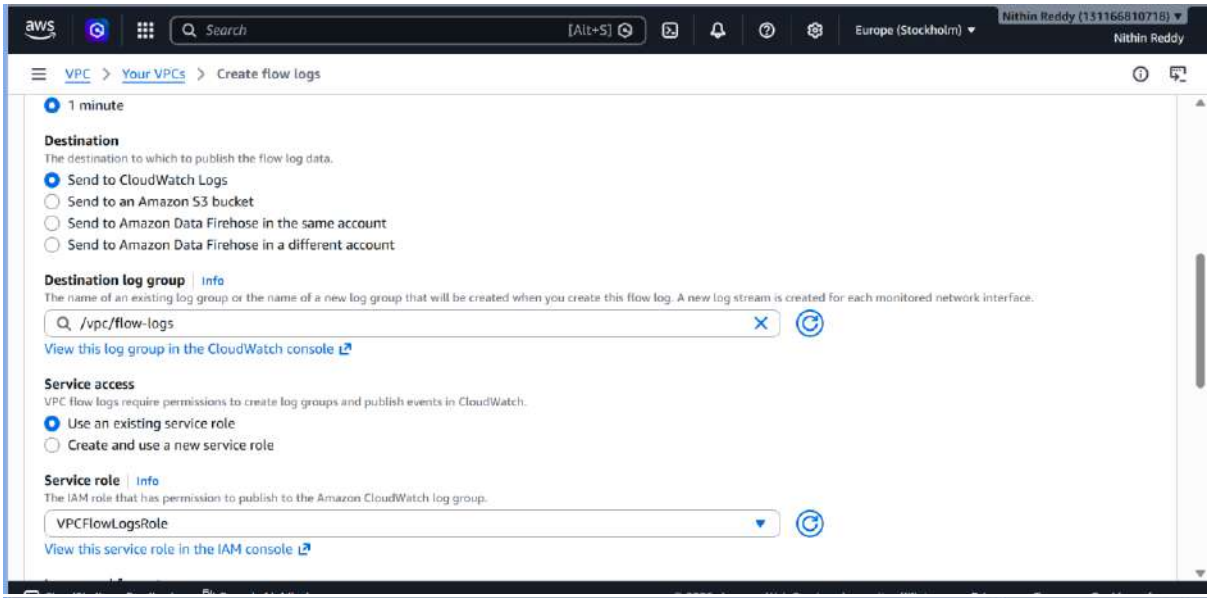
STEP 6 — Create Flow Log → CloudWatch

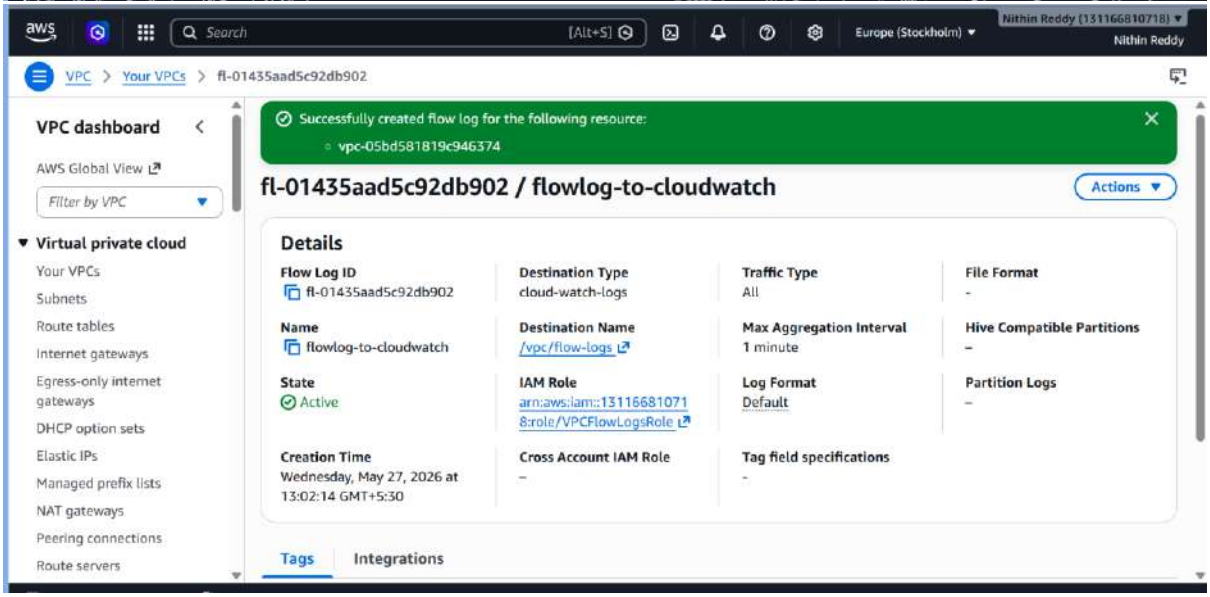
Still on same VPC → **Flow logs tab → Create flow log** again



Still on same VPC → **Flow logs tab → Create flow log** again

Field	**Value**
Name	flowlog-to-cloudwatch
Filter	**All**
Maximum aggregation interval	**1 minute**
Destination	**Send to CloudWatch Logs**
Destination log group	/vpc/flow-logs
IAM role	VPCFlowLogsRole
Log record format	**AWS default format**





What Each Step Did & Why

Step	What You Did	Why You Did It
Created S3 bucket	Made a storage container for log files	Flow logs need somewhere to write .gz files
Added bucket policy	Gave AWS permission to write into your bucket	Without this, AWS delivery service gets "Access Denied"

Step	What You Did	Why You Did It
Created CloudWatch Log Group	Made a folder in CloudWatch for log streams	Flow logs need a destination group to write real-time records
Created IAM Role	Gave VPC Flow Logs permission to talk to CloudWatch	CloudWatch won't accept logs from an unauthorized caller
Created Flow Log → S3	Turned on traffic recording to S3	Captures all ACCEPT + REJECT traffic, stored as files
Created Flow Log → CloudWatch	Turned on traffic recording to CloudWatch	Same traffic, but now queryable in real time
Generated traffic	Ran curl/ping from EC2	Empty traffic = empty logs — needed to create real records
Created Metric Filter	Filtered REJECT records into a metric	Turns raw logs into a number CloudWatch can alarm on

✅ Validation

Flow Logs Status

- Go to VPC → Your VPC → Flow logs tab
- Both flowlog-to-s3 and flowlog-to-cloudwatch must show **Active**

CloudWatch

- Go to CloudWatch → Log groups → /vpc/flow-logs
- Log streams named like eni-0abc1234... must be visible
- Click any stream → records must show ACCEPT or REJECT entries

S3

- Go to S3 → your bucket → AWSLogs/131166810718/vpcflowlogs/region/year/month/day
- .gz files must be present inside the date folder

IAM Role

- Go to IAM → Roles → VPCFlowLogsRole → Trust relationships
- Must show vpc-flow-logs.amazonaws.com

Troubleshooting

Flow log shows Failed

- IAM trust policy has wrong service name
- Fix: must be exactly vpc-flow-logs.amazonaws.com

S3 flow log Failed

- Bucket policy has wrong account ID or ARN
- Fix: remove < > brackets, check account ID is 131166810718

CloudWatch streams empty

- No traffic generated after enabling logs
- Fix: run curl http://localhost:8080/sample/ and ping -c 5 8.8.8.8 from EC2, wait 5 mins

S3 files not appearing

- S3 delivery takes up to 15 minutes — normal delay
- Fix: wait and navigate deep into AWSLogs/.../day/ folder

Log group missing in dropdown

- Log group and VPC are in different regions
- Fix: create log group in same region as your VPC

Bucket policy save error

- JSON syntax error in the policy
- Fix: check for missing , between statements, missing { or }, no < > around account ID

No REJECT records in logs

- No blocked traffic happened yet
- Fix: try connecting to a closed port to generate reject traffic